

ICAI

Proyecto: Bases de Datos

Rodrigo Sicilia y Claudia Moya

Segundo de Carrera - Curso 2025/2026

1. Primera parte: diseño y carga de datos

1.1. Objetivo de esta primera parte

En esta primera parte del proyecto se ha diseñado la forma en la que se van a almacenar los datos de las reviews utilizando dos tecnologías diferentes, MySQL y MongoDB. El objetivo no es solo guardar la información, sino hacerlo de una manera coherente, justificada y útil para las siguientes partes del trabajo.

Para ello, el diseño debía cumplir varias condiciones importantes. Por un lado, era necesario repartir la información entre una base de datos relacional y una no relacional. Por otro lado, había que asegurarse de que todos los campos originales de los archivos JSON quedarán almacenados entre ambas bases de datos. Además, se permitía añadir información nueva si ayudaba a organizar mejor los datos, como por ejemplo identificadores internos. También era importante tener en cuenta que un mismo usuario podía aparecer en varios de los ficheros de entrada, por lo que no convenía tratar cada archivo como un conjunto completamente independiente. Por último, el campo `reviewTime` debía almacenarse como una fecha real y no como texto.

Teniendo en cuenta todo esto, se ha planteado un diseño híbrido en el que MySQL se utiliza para la parte más estructurada de la información y MongoDB para la parte más documental.

1.2. Decisión general: qué se guarda en MySQL y qué se guarda en MongoDB

La idea principal del diseño ha sido separar los datos según su naturaleza.

En MySQL se ha guardado la información que encaja mejor en un modelo relacional, es decir, aquella que representa entidades claras y relaciones entre ellas. Aquí entran los usuarios, los artículos, las categorías, la parte estructurada de las reviews y también la procedencia categórica de cada review, que en el modelo final se representa mediante la tabla `REVIEW_CATEGORIA`.

En MongoDB se ha guardado la parte menos estructurada de cada review, especialmente los campos de texto y el campo *helpful*. Esta decisión tiene sentido porque MongoDB trabaja muy bien con documentos y con estructuras flexibles, mientras que MySQL resulta más adecuado cuando hay relaciones bien definidas y se quiere mantener integridad entre tablas.

1.3. Justificación del diseño en MySQL

Para la parte estructurada del proyecto se ha optado por utilizar MySQL. La idea es almacenar ahí la información que tiene una organización clara y que, además, va a ser necesaria para hacer consultas, relaciones entre entidades y análisis posteriores. En

nuestro caso, esto incluye los *usuarios*, los *artículos*, las *categorías*, la parte principal de cada *review* y la procedencia categórica de cada una de ellas.

La elección de MySQL tiene sentido porque permite trabajar con tablas relacionadas entre sí, definir claves primarias y foráneas, y establecer restricciones que ayudan a mantener la consistencia de los datos. Esto es especialmente útil en un proyecto como este, ya que después va a ser necesario hacer consultas como contar reviews por categoría, calcular medias de puntuación, relacionar usuarios con artículos o estudiar cómo evolucionan las valoraciones con el tiempo.

Desde el principio, una de las ideas principales del diseño fue evitar duplicados innecesarios. Por ejemplo, un mismo usuario puede aparecer en distintos ficheros JSON, así que no tendría sentido almacenarlo varias veces. Lo mismo ocurre con los artículos, que aunque aparezcan en muchas reviews, lo correcto es representarlos una sola vez y relacionarlos después con las reviews correspondientes. Por eso MySQL encaja bien en esta parte del trabajo, ya que permite separar correctamente cada entidad y conectar unas con otras de forma ordenada.

Sin embargo, antes de cerrar definitivamente el modelo, vimos que no bastaba con diseñarlo teóricamente, sino que era importante comprobar si realmente se ajustaba a los datos originales. Para eso hicimos varias comprobaciones directamente sobre los archivos JSON, utilizando scripts de Python que recorren los ficheros línea a línea. Esta forma de trabajar nos permitió entender mucho mejor cómo eran los datos de verdad y detectar algunos casos que no se veían o entendían a simple vista.

1.3.1. Diseño inicial

En una primera versión del modelo se asumió que cada artículo pertenecía a una sola categoría. Con esa hipótesis, la categoría utilizada en el análisis se deducía a partir del artículo, lo que llevaba naturalmente a una relación simple entre CATEGORIA y ARTICULO.

Sin embargo, no se usó este modelo ya que antes de fijar el modelo definitivo, se realizaron comprobaciones empíricas directamente sobre los archivos JSON mediante scripts auxiliares en Python. Esta decisión está alineada con el propio enunciado del proyecto, que exige trabajar con Python y no cargar los ficheros completos en memoria. Por ello, los scripts se diseñaron para recorrer los datasets línea a línea y validar con datos reales si las hipótesis del modelo eran correctas.

Todas estas comprobaciones están implementadas en el fichero *verificar_datasets.py*, que recorre los cuatro ficheros JSON línea a línea (sin cargarlos en memoria) y genera ficheros CSV con los resultados detallados de cada análisis. De esta forma, los hallazgos son fácilmente reproducibles y verificables.

1.3.2. Primera corrección: artículos multicategoría

Al analizar los JSON vimos que un mismo *asin* puede aparecer en varios datasets, por lo que un artículo puede estar asociado a varias categorías. En un primer momento se consideró una tabla intermedia ARTICULO_CATEGORIA, pero como se explica en el

apartado siguiente, esa solución no era suficiente para las consultas por categoría a nivel de review.

1.3.3. Segunda corrección: la categoría debe asociarse a la review

El problema importante apareció al observar cómo se comportaban las consultas por categoría. Si la categoría se deduce desde el artículo, una review escrita en el JSON de una categoría puede llegar a aparecer al consultar otra categoría distinta, simplemente porque el artículo esté presente también en ese otro dataset, y eso produce una atribución incorrecta de las reviews.

Por lo tanto, la idea clave del modelo final es que la categoría que importa para el análisis no debe asociarse al *artículo*, sino a la procedencia de la *review*.

Pero en este punto es importante distinguir dos cosas. Por un lado, está la ocurrencia física de una línea dentro de un fichero JSON concreto, y por otro lado, está la *review* lógica que se almacena en la base de datos tras eliminar duplicados exactos. Esa distinción es necesaria porque una misma *review* lógica puede aparecer repetida en varios ficheros distintos.

Para comprobarlo, se desarrolló un script auxiliar en Python que recorrió los cuatro datasets línea a línea, sin cargar los ficheros completos en memoria, y agrupó las reviews por *reviewerID*, *asin* y *unixReviewTime*. Después, para cada grupo repetido, se comparó el contenido completo mediante hash con el fin de distinguir entre repeticiones exactas y posibles conflictos, y este análisis generó además varios CSV de apoyo para revisar los resultados.

Los resultados obtenidos mostraron que sí existen reviews repetidas entre distintos ficheros. En concreto, aparecieron 387 casos de reviews presentes en más de un fichero, y todas ellas eran repeticiones exactas, aunque no se detectó ningún caso conflictivo con la misma tripleta y contenido distinto. Esto nos demuestra que no es correcto afirmar que cada *review* lógica pertenezca necesariamente a un único dataset.

Por tanto, el modelo no debe guardar una copia independiente de la review por cada fichero en el que aparece, pero sí debe conservar la trazabilidad de en qué categorías o datasets fue encontrada. Para resolverlo se creó la tabla `REVIEW_CATEGORIA(id_review, id_categoria)`. De este modo, `REVIEW` almacena una sola vez la review lógica duplicada, mientras que `REVIEW_CATEGORIA` registra todas las categorías en las que esa review apareció en los datos de origen.

A partir de ese momento, todas las consultas por categoría deben basarse en `REVIEW_CATEGORIA`, ya que es la estructura que permite que cada consulta utilice la procedencia real de las reviews y evite atribuciones incorrectas derivadas de la multicategoría del artículo.

Por tanto, `ARTICULO_CATEGORIA` se elimina del modelo final y la única tabla que conecta categorías con el resto del esquema es `REVIEW_CATEGORIA`.

1.3.4. Modelo final adoptado

Con esta corrección, el núcleo del modelo relacional final queda formado por las tablas USUARIO, CATEGORIA, ARTICULO, REVIEW y REVIEW_CATEGORIA. Así, entre USUARIO y REVIEW hay una relación de 1 a N, entre ARTICULO y REVIEW también hay una relación de 1 a N, y entre REVIEW y CATEGORIA hay una relación M a N gestionada mediante REVIEW_CATEGORIA. La tabla ARTICULO_CATEGORIA no forma parte del modelo final, ya que toda la información de procedencia categórica se gestiona exclusivamente a nivel de review.

Además de esta evolución del modelo, se hicieron otras comprobaciones para validar decisiones adicionales del diseño. Una de ellas fue analizar si había reviews duplicadas. Para ello se tomó como referencia la combinación reviewerID, asin y unixReviewTime, y se revisó si esa tripleta se repetía en los datos. Aparecieron varios casos repetidos, pero todos correspondían a duplicados exactos, es decir, a la misma review repetida, no a reviews distintas compartiendo esa misma combinación de valores, lo que confirma que esa combinación sigue siendo una buena base para identificar duplicados y justificó mantener en la tabla REVIEW una restricción de unicidad sobre id_usuario, id_articulo y unixReviewTime.

A nivel práctico, esta conclusión implica que durante la carga de datos habrá que evitar insertar dos veces la misma review. Si una review ya ha sido registrada con esa combinación de usuario, artículo e instante temporal, no debe volver a insertarse. Así se evita introducir redundancia en la base de datos relacional y también posibles incoherencias con la parte almacenada en MongoDB.

Otra comprobación se centró en los datos del usuario, concretamente en reviewerID y reviewerName. Al revisar los JSON vimos que reviewerID sí se comporta como un identificador bastante estable, pero reviewerName no. En muchos casos un mismo reviewerID aparece unas veces con nombre y otras veces sin él, y además hay algunos casos en los que el mismo reviewerID aparece asociado a nombres distintos. Esto nos llevó a la conclusión de que reviewerName no se puede tratar como un dato totalmente fiable para identificar usuarios, sino más bien como una información descriptiva que puede faltar o variar.

Por eso se decidió mantener reviewerID como identificador único del usuario y dejar reviewerName como un atributo auxiliar. Es decir, la tabla USUARIO sigue teniendo sentido, pero el nombre del usuario no se interpreta como un campo estable ni como una clave del modelo. En la carga de datos, si de un usuario no tenemos nombre y aparece uno no vacío, se guarda, si ya hay un nombre no vacío, no se sustituye por uno vacío, y si aparecen varias versiones no vacías, se conservará la primera registrada. De esta forma se mantiene el modelo simple, sin añadir complejidad innecesaria por un dato que en origen no es completamente consistente.

Además, se revisaron otros campos como reviewTime y helpful para comprobar si presentaban problemas de formato o anomalías importantes. En estas comprobaciones no aparecieron incidencias relevantes que obligaran a modificar el diseño, así que se mantuvo la decisión inicial, que era guardar reviewTime en MySQL, ya que forma parte de la

información estructurada de la review, y almacenar `helpful` junto con `summary` y `reviewText` en MongoDB, donde encaja mejor por su carácter más flexible.

En definitiva, el diseño en MySQL no se decidió solo por una cuestión teórica, sino también a partir de una revisión real de los datos originales. Primero se detectó la multicategoría de los artículos, después se comprobó que eso no bastaba para analizar correctamente las reviews por categoría, y finalmente se adoptó un modelo en el que la categoría relevante queda asociada a cada review mediante `REVIEW_CATEGORIA`. Gracias a ello, el diseño final representa mucho mejor la estructura real de los datos con los que se va a trabajar en el proyecto.

1.4. Descripción de las tablas de MySQL

1.4.1. Tabla USUARIO

La tabla `USUARIO` se ha creado para representar a los autores de las reviews.

En los archivos originales aparece el campo `reviewerID`, que identifica al usuario, y también `reviewerName`, que almacena su nombre. Sin embargo, en lugar de usar `reviewerID` como clave primaria de la tabla, se ha añadido un identificador interno llamado `id_usuario`, que es autoincremental.

Esta decisión se ha tomado porque trabajar con un entero como clave primaria simplifica mucho las relaciones con otras tablas. Además, permite que las claves foráneas que lo referencien sean más compactas y manejables que si se utilizara directamente una cadena de texto como `reviewerID`. Esto hace el diseño más limpio, uniforme y eficiente.

Aun así, `reviewerID` no se pierde, sino que se conserva como identificador original del usuario y se le impone una restricción `UNIQUE` para evitar repeticiones. De esta forma, cada usuario queda representado de manera única, incluso aunque aparezca en varios conjuntos de datos.

1.4.2. Tabla CATEGORIA

La tabla `CATEGORIA` se ha incluido para representar los distintos tipos de producto con los que se trabaja en el proyecto. En la carga inicial se insertan las siguientes categorías:

- Toys and Games
- Video Games
- Digital Music
- Musical Instruments

En el modelo final, estas categorías no deben entenderse como una propiedad intrínseca del artículo que se usa directamente en las consultas, sino como la forma de representar el origen de las reviews. Es decir, sirven para identificar de qué dataset procede cada review y poder analizar correctamente la información por categoría.

También aquí se ha añadido una clave primaria interna, *id_categoria*. Esta decisión permite mantener uniformidad con el resto del diseño y hacer que, cuando se utiliza como referencia en otras tablas, se trabaje con un valor numérico más compacto y manejable que un texto. Además, *nombre_categoria* se define como único para evitar categorías duplicadas.

1.4.3. Tabla ARTICULO

La tabla ARTICULO se utiliza para almacenar los productos sobre los que los usuarios escriben reviews.

En los datos originales, cada artículo se identifica mediante el campo *asin*. Ese campo se conserva, pero igual que en las demás tablas, se ha añadido un identificador interno llamado *id_articulo* como clave primaria.

Se ha optado por esta solución porque mantiene el diseño uniforme y facilita las relaciones con la tabla REVIEW. Así, internamente se trabaja con claves numéricas y se conserva *asin* como identificador original del producto. Además, el hecho de utilizar un identificador interno hace que las claves foráneas ocupen menos y sean más cómodas de manejar que si se utilizara directamente el valor textual de *asin* como referencia.

Además, se impone la restricción UNIQUE(*asin*) para garantizar que cada artículo quede representado una única vez en toda la base de datos, ya que *asin* es el identificador original del producto en los datos de entrada.

En el modelo final, ARTICULO identifica el producto y la categoría no se asocia al artículo, sino a cada review mediante REVIEW_CATEGORIA.

1.4.4. Tabla REVIEW

La tabla REVIEW recoge la parte estructurada de cada review.

En ella se almacena qué usuario ha escrito la review, sobre qué artículo la ha hecho, qué puntuación ha dado y en qué momento se realizó. Esta tabla actúa, por tanto, como el núcleo del modelo, ya que conecta a usuarios y artículos a través de las valoraciones.

Se ha añadido un identificador interno *id_review* como clave primaria autoincremental. Esta decisión es especialmente importante porque una review necesita tener identidad propia. No basta con identificar al usuario o al artículo, ya que un mismo usuario puede hacer varias reviews y un mismo artículo puede recibir muchas. Por eso cada review debe tener su propio identificador único.

Además, ese mismo *id_review* será el que después se utilizará como puente entre MySQL y MongoDB. Gracias a ello, la parte estructurada de la review se guarda en MySQL y la parte documental se guarda en MongoDB, pero ambas siguen estando conectadas de forma unívoca.

Dentro de esta tabla se guarda también el campo *overall*, porque es una puntuación numérica que se utilizará directamente en análisis posteriores, se guarda como TINYINT, ya que siempre es un número entero pequeño. También se conserva *unixReviewTime*, ya que

resulta útil para trabajar con la dimensión temporal. En cuanto a *reviewTime*, se almacena como DATE, cumpliendo así el requisito de no guardarlo como string.

Por último, se añade una restricción UNIQUE(id_usuario, id_articulo, unixReviewTime) para evitar la inserción accidental de duplicados exactos.

La categoría de la review no se deduce desde ARTICULO, pues en el modelo final, la parte estructurada de la review se guarda en REVIEW y su procedencia categórica se representa mediante la tabla REVIEW_CATEGORIA.

1.4.5. Tabla REVIEW_CATEGORIA

La tabla REVIEW_CATEGORIA se ha creado para representar explícitamente la procedencia categórica de cada review.

Sus campos son *id_review* e *id_categoria*. Ambos forman una clave primaria compuesta y, además, actúan como claves foráneas hacia REVIEW y CATEGORIA, respectivamente.

La razón de crear esta tabla es corregir el problema semántico detectado al asociar la categoría directamente al artículo. Si la categoría se deduce desde ARTICULO, una review puede aparecer en una categoría distinta de aquella en la que realmente fue publicada. En cambio, al registrar la categoría a nivel de review, cada valoración queda vinculada a su dataset de origen.

Gracias a esta solución, una review solo cuenta en la categoría donde realmente apareció, que es justo el comportamiento que necesitan las consultas del proyecto. Por eso REVIEW_CATEGORIA es preferible a deducir la categoría desde el artículo cuando el análisis se hace a nivel de review.

Además, esta tabla no solo corrige el problema de deducir la categoría desde ARTICULO, sino que también permite compatibilizar dos decisiones del diseño que son esenciales: deduplicar reviews exactas en la tabla REVIEW y conservar, al mismo tiempo, la información sobre todos los datasets en los que cada review fue encontrada. Gracias a ello, una misma review lógica puede almacenarse una sola vez y seguir quedando asociada a varias categorías de procedencia cuando así ocurra en los datos originales.

1.5. Índices

1.5.1. Índices implícitos

Una parte de los índices del modelo aparece de forma implícita al definir claves primarias y restricciones UNIQUE. En este grupo están los índices asociados a las claves primarias de USUARIO(id_usuario), CATEGORIA(id_categoria), ARTICULO(id_articulo) y REVIEW(id_review), así como los derivados de UNIQUE(reviewerID), UNIQUE(nombre_categoria), UNIQUE(asin) y UNIQUE(id_usuario, id_articulo, unixReviewTime) en REVIEW.

En `REVIEW_CATEGORIA`, la clave primaria compuesta (`id_review`, `id_categoria`) evita duplicar la misma relación entre una review y una categoría y proporciona un acceso indexado en ese orden de columnas.

Además, MySQL crea automáticamente un índice sobre cada columna que actúa como clave foránea. Esto incluye `REVIEW(id_usuario)`, `REVIEW(id_articulo)` y `REVIEW_CATEGORIA(id_categoria)`. Por ello no se declaran explícitamente en el código, pues ya existen como consecuencia de las restricciones de integridad referencial.

1.5.2. Índices añadidos manualmente

Además de los anteriores, conviene añadir algunos índices específicos para mejorar el rendimiento de las consultas más importantes del proyecto.

`idx_review_categoria_categoria_review` sobre `REVIEW_CATEGORIA(id_categoria, id_review)`, es el índice más importante del nuevo modelo, porque las consultas por categoría filtran primero por `id_categoria` y después enlazan con `REVIEW`. La clave primaria de `REVIEW_CATEGORIA` está ordenada como (`id_review`, `id_categoria`), así que este índice adicional cubre el patrón de acceso inverso que realmente usan las consultas analíticas.

`idx_review_reviewTime` sobre `REVIEW(reviewTime)` resulta útil para las consultas que agrupan reviews por fecha o por año, ya que facilita los recorridos ordenados por la componente temporal almacenada como `DATE`.

`idx_review_unix_review_time` sobre `REVIEW(unixReviewTime)` mejora las consultas de evolución temporal y las agregaciones que trabajan directamente con la marca de tiempo original.

`idx_usuario_reviewer_name` sobre `USUARIO(reviewerName)` ha sido creado específicamente para la consulta de la *sección 4.3*, que filtra los usuarios con *reviewerName* no vacío (`IS NOT NULL AND != ""`) y ordena los resultados por ese campo. Sin este índice, MySQL realizaría un recorrido completo de la tabla `USUARIO` para filtrar y ordenar, lo que sería bastante costoso dado el volumen de usuarios.

No se considera necesario crear un índice específico sobre `REVIEW(overall)`, ya que su cardinalidad es muy baja y su utilidad sería bastante limitada teniendo en cuenta el coste de crear el índice.

En MongoDB, la colección `reviews_texto` debe tener un índice `UNIQUE` sobre el campo `id_review`. Este índice permite enlazar de forma eficiente cada documento con su fila correspondiente en MySQL y, además, actúa como protección frente a duplicados en la colección documental.

1.6. Relaciones entre las tablas

Las relaciones principales del modelo final son las siguientes:

- Un usuario puede escribir muchas reviews.
- Un artículo puede recibir muchas reviews.
- Una review queda asociada a la categoría de la que procede mediante REVIEW_CATEGORIA.

Esto significa que:

- Entre USUARIO y REVIEW hay una relación 1 a N.
- Entre ARTICULO y REVIEW hay una relación 1 a N.
- Entre REVIEW y CATEGORIA hay una relación M a N, gestionada mediante la tabla intermedia REVIEW_CATEGORIA.

Estas relaciones reflejan de forma más precisa cómo deben interpretarse los datos del proyecto. Gracias a esta estructura, el modelo evita atribuciones incorrectas de reviews entre categorías y permite hacer consultas consistentes con el dataset real de origen.

1.7. Esquema relacional

El esquema relacional final del modelo es el siguiente:

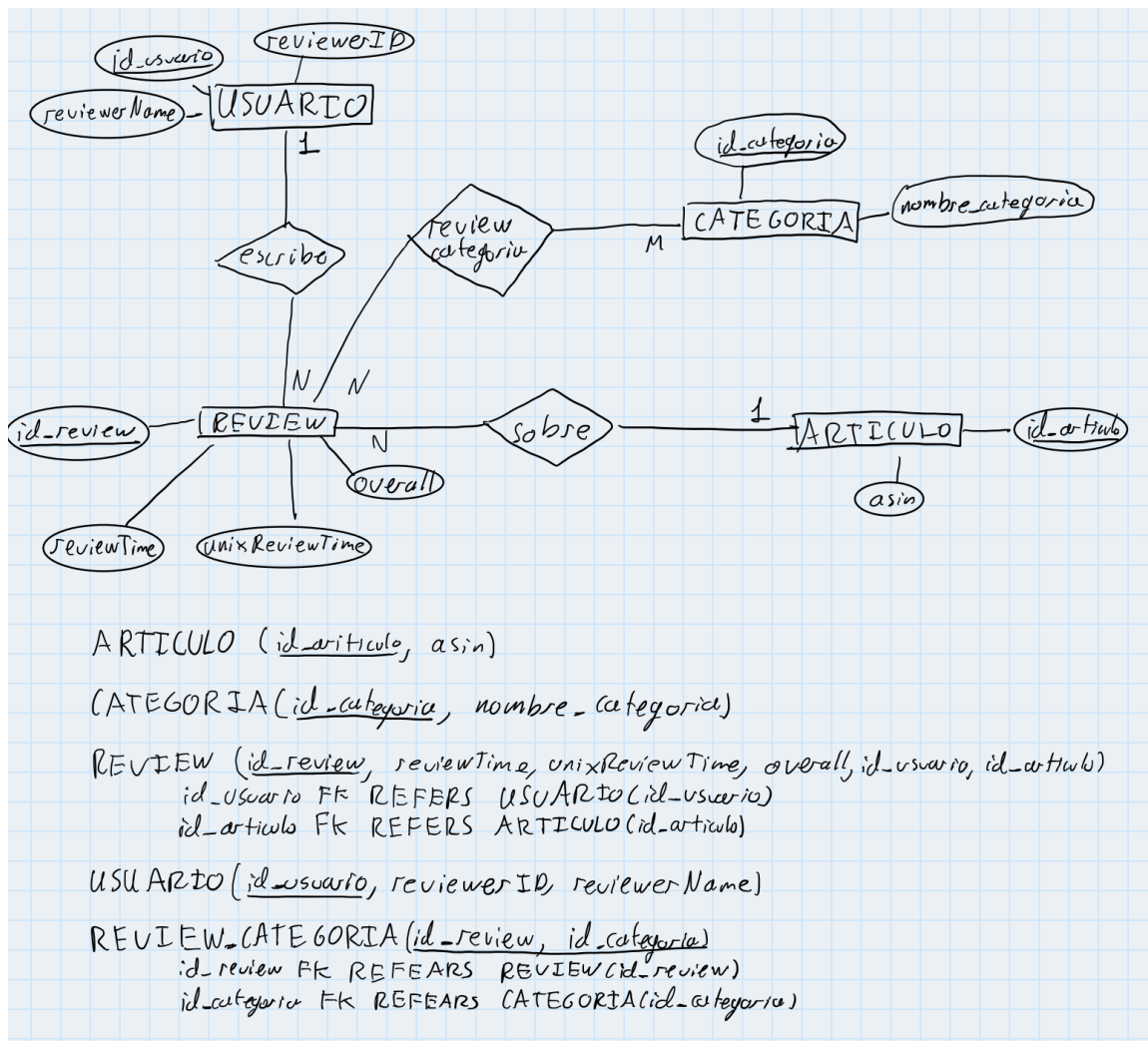


Figura 1.1. Esquema relacional final del proyecto.

1.8. Justificación del diseño en MongoDB

En MongoDB se ha decidido almacenar la parte más documental de cada review, es decir, la información que no necesita descomponerse en varias tablas y que encaja mejor en formato documento.

En concreto, se ha creado una colección llamada *reviews_texto*. Cada documento de esta colección contiene los campos *helpful*, *summary* y *reviewText*, además de un campo *id_review* que permite enlazar el documento con su correspondiente fila en la tabla REVIEW de MySQL.

Esta decisión está bien justificada por la propia naturaleza de los datos. Los campos *summary* y *reviewText* son textos de longitud variable, por lo que encajan de forma muy natural en una base de datos documental. Por su parte, *helpful* aparece como un array de dos enteros, y MongoDB permite almacenar este tipo de estructura directamente, sin necesidad de transformarla ni dividirla en varias columnas.

Por tanto, MongoDB se utiliza aquí para guardar precisamente aquello que tiene más sentido modelar como documento completo y no como estructura relacional.

1.9. Estructura de los documentos en MongoDB

La colección creada en MongoDB es *reviews_texto*, y la estructura general de cada documento es la siguiente:

```
{
  "_id": ObjectId("..."),
  "id_review": 12345,
  "helpful": [8, 12],
  "summary": "Great game",
  "reviewText": "I bought this for my son and he loved it..."
}
```

En esta estructura, el campo *_id* es el identificador interno propio de MongoDB, y el campo *id_review* coincide con *id_review* en MySQL, por lo que sirve como enlace entre ambas bases de datos. Gracias a este campo, se puede relacionar sin problemas la parte documental almacenada en MongoDB con la parte estructurada almacenada en MySQL.

1.10. Reparto final de los campos entre ambas bases de datos

En MySQL se almacenan los campos originales *reviewerID*, *reviewerName*, *asin*, *overall*, *unixReviewTime* y *reviewTime*. Además, en MySQL se representa la procedencia categórica de cada review mediante la tabla REVIEW_CATEGORIA, y también se añaden los identificadores internos *id_usuario*, *id_categoria*, *id_articulo* e *id_review*.

En MongoDB se almacenan *helpful*, *summary* y *reviewText*, y además se añade *id_review* como campo de enlace con la tabla REVIEW de MySQL.

De esta manera, todos los campos originales de los JSON quedan guardados entre las dos bases de datos y, al mismo tiempo, se añaden los identificadores necesarios para que el diseño sea más claro y funcional.

1.11. Optimización de la carga de datos

Una vez definido el modelo final, se revisó el rendimiento del script de carga (*load_data.py*), ya que el tiempo de ejecución era excesivo para el volumen de datos del proyecto.

El problema principal era el número de consultas individuales que se realizaban a MySQL por cada review, porque para cada línea del JSON, el script consultaba la tabla USUARIO para obtener el identificador del usuario, la tabla ARTICULO para obtener el del artículo, la tabla REVIEW para detectar duplicados y la tabla REVIEW_CATEGORIA para registrar la procedencia. Además, en cada consulta de existencia primero se buscaba la fila y luego se insertaba si no existía, por lo que doblábamos el número de operaciones (en total, entre 6 y

8 consultas por review). Con cientos de miles de reviews, esto suponía millones de consultas a la base de datos.

Para resolverlo se aplicaron varias optimizaciones que no modifican el modelo ni el resultado final de la carga.

En primer lugar, se introdujeron cachés en memoria mediante diccionarios de Python para las tablas USUARIO y ARTICULO. La primera vez que aparece un *reviewerID* o un *asin* se consulta MySQL y se guarda el resultado en el diccionario, y las siguientes apariciones del mismo identificador se resuelven directamente en memoria, sin tocar la base de datos. Esto elimina la gran mayoría de SELECTs repetidos, ya que muchos usuarios y artículos aparecen en incluso cientos de reviews.

En segundo lugar, se sustituyó el patrón SELECT-then-INSERT por INSERT IGNORE en los dos puntos donde se detectaban duplicados, los cuales eran REVIEW_CATEGORIA y REVIEW.

En REVIEW_CATEGORIA el cambio es directo, porque la clave candidata compuesta (id_review, id_categoria) ya garantiza que no se insertan duplicados, así que basta con intentar la inserción y dejar que MySQL la descarte silenciosamente si ya existe, lo que reduce dos consultas a una sola.

En REVIEW la idea es parecida pero un poco más compleja, aquí no basta con detectar el duplicado, sino que además hay que recuperar el id_review que ya existe para poder enlazar la review con REVIEW_CATEGORIA y con MongoDB. Para ello se ha implementado la función insertar_o_recuperar_id_review, que intenta un INSERT IGNORE sobre REVIEW apoyándose en la restricción UNIQUE(id_usuario, id_articulo, unixReviewTime). Si MySQL acepta la inserción (cursor.rowcount igual a 1), se devuelve directamente lastrowid como id_review. Si MySQL la descarta porque ya existía, lo que generaría un rowcount igual a 0, se lanza un único SELECT de fallback para recuperar el identificador ya guardado. De este modo, en el caso habitual de review nueva solo se ejecuta una consulta, y solo en los duplicados entre datasets (387 casos reales en los ficheros del proyecto) se hacen dos. Este diseño sustituye al patrón anterior, en el que se hacía siempre un SELECT previo para ver si la review existía y después, si no existía, un INSERT posterior. Ahora esto es más rápido.

Por último, se agruparon las inserciones en MongoDB por lotes de 10.000 documentos utilizando insert_many en lugar de insert_one. Esto reduce los round-trips de red con MongoDB de uno por review a uno por lote. Si algún documento del lote falla, se identifican los documentos fallidos y se eliminan las filas correspondientes de MySQL para mantener la sincronización entre ambas bases.

1.12. Fichero de configuración

Toda la configuración necesaria para conectarse a las bases de datos y localizar los ficheros JSON se centraliza en el fichero *configuracion.py*. En él se definen las credenciales y parámetros de conexión de MySQL, MongoDB y Neo4J, así como las rutas de los cuatro datasets iniciales. Para ejecutar el proyecto en otro entorno, basta con modificar únicamente este fichero, sin necesidad de tocar el resto del código.

2. Segunda parte: aplicación Python para el acceso y visualización de los datos

2.1. Objetivo de esta segunda parte

En esta segunda parte del proyecto se ha desarrollado un programa en Python llamado *menu_visualizacion.py*, cuyo objetivo es permitir la exploración de los datos mediante varias representaciones gráficas, tal y como pide el enunciado.

Para resolver este apartado nosotros hemos optado por un menú por terminal, porque creemos que resulta más sencillo de mantener, permite separar muy bien cada funcionalidad y hace que el flujo de ejecución sea mucho más claro para el usuario.

La idea general de esta parte no ha sido únicamente dibujar gráficas, sino hacerlo de forma coherente con el modelo de datos adoptado en la primera parte. Por eso, en todas las consultas donde interviene la categoría, se utiliza `REVIEW_CATEGORIA` para obtener la review, evitando deducir la categoría desde el artículo.

2.2. Decisiones generales de diseño

La aplicación se ha implementado en un único fichero, *menu_visualizacion.py*, y utiliza tanto MySQL como MongoDB.

MySQL se usa para todas las consultas estructuradas, es decir, conteos, agrupaciones por año, histogramas por nota, popularidad de artículos, evolución temporal y número de reviews por usuario. Sin embargo, MongoDB se utiliza en la nube de palabras, porque ahí interesa recuperar el campo *summary*, que forma parte de la parte documental de las reviews.

Para facilitar la reutilización del programa con distintas configuraciones, las conexiones no dependen de nombres rígidos escritos dentro del código, sino que los parámetros de conexión a MySQL y MongoDB se importan directamente desde *configuracion.py*, por lo que el programa no tiene una configuración concreta y es más fácil de reutilizar. Este mismo modo de importación se utiliza en todos los módulos del proyecto (*load_data.py*, *menu_visualizacion.py*, *neo4JProyecto.py*, *inserta_dataset.py* y *recomendacion.py*).

También se ha tomado la decisión de validar directamente contra la base de datos las categorías que introduce el usuario ya que es mejor que mantener una lista fija en Python, porque hace que el programa siga siendo correcto aunque más adelante se inserten nuevos

datasets en la tabla CATEGORIA. Así, la base de datos actúa como fuente real y el menú se adapta automáticamente al contenido disponible.

En concreto, todos los módulos interactivos del proyecto (*menu_visualizacion.py*, *neo4JProyecto.py* y *recomendacion.py*) implementan una función llamada *obtener_categorias*, que ejecuta una única consulta SQL para recuperar la lista completa de categorías disponibles en la tabla CATEGORIA. Esta lista se muestra al usuario antes de solicitarle que elija una categoría, lo que mejora la usabilidad al no tener que adivinar los nombres exactos. En cuanto a la validación, se realiza comprobando si la entrada del usuario pertenece a dicha lista (mediante el operador `in` de Python), en lugar de ejecutar una consulta SQL por cada intento, lo cual reduce el número de accesos a la base de datos y simplifica la lógica del programa.

Además, se han incorporado varias comprobaciones de errores y mensajes claros para evitar comportamientos bruscos. Por ejemplo, si una opción no es válida, si una categoría no existe, si no hay datos para una consulta concreta o si el usuario interrumpe una operación, el programa responde con un mensaje comprensible en lugar de fallar y ya está.

2.3. Estructura general del menú

El programa presenta un menú por terminal con siete opciones de visualización y una opción de salida. Tras ejecutar una opción, el menú vuelve a mostrarse hasta que el usuario decide terminar el programa.

Las opciones implementadas son las siguientes:

- evolución de reviews por años
- evolución de la popularidad de los artículos
- histograma por nota
- evolución de las reviews a lo largo del tiempo
- histograma de reviews por usuario
- nube de palabras por categoría
- visualización extra: media de nota por categoría

Esta organización sigue directamente la estructura del enunciado, aunque hemos añadido una pequeña ampliación en la opción de evolución temporal, donde no solo se ofrece la visualización por categorías separadas, sino también una sola categoría o todas juntas, lo que hace que la herramienta sea más flexible y útil para analizar los datos.

2.4. Evolución de reviews por años

La primera visualización muestra el número de reviews por año en formato histograma.

Para esta opción, el programa pide al usuario una categoría concreta o la opción Todo. Si el usuario elige Todo, la consulta se hace directamente sobre la tabla REVIEW. En cambio, si elige una categoría concreta, la consulta se realiza enlazando REVIEW con REVIEW_CATEGORIA y CATEGORIA, de forma que solo se cuenten las reviews cuya procedencia pertenece realmente a esa categoría.

En esta visualización se utiliza *reviewTime* y se agrupa por YEAR(*reviewTime*). Además, se excluyen las filas donde *reviewTime* es NULL, lo cual es necesario porque no tendría sentido intentar extraer el año de una fecha inexistente, y además así se evita introducir errores o conteos artificiales en la gráfica.

Se ha escogido un histograma porque es la representación más natural cuando se quiere comparar cuántas reviews hay en cada año (aparte de que es la pedida en el enunciado). También se fuerzan los años en el eje x para que la lectura de la gráfica sea más clara y no dependan de un espaciado automático poco estable.

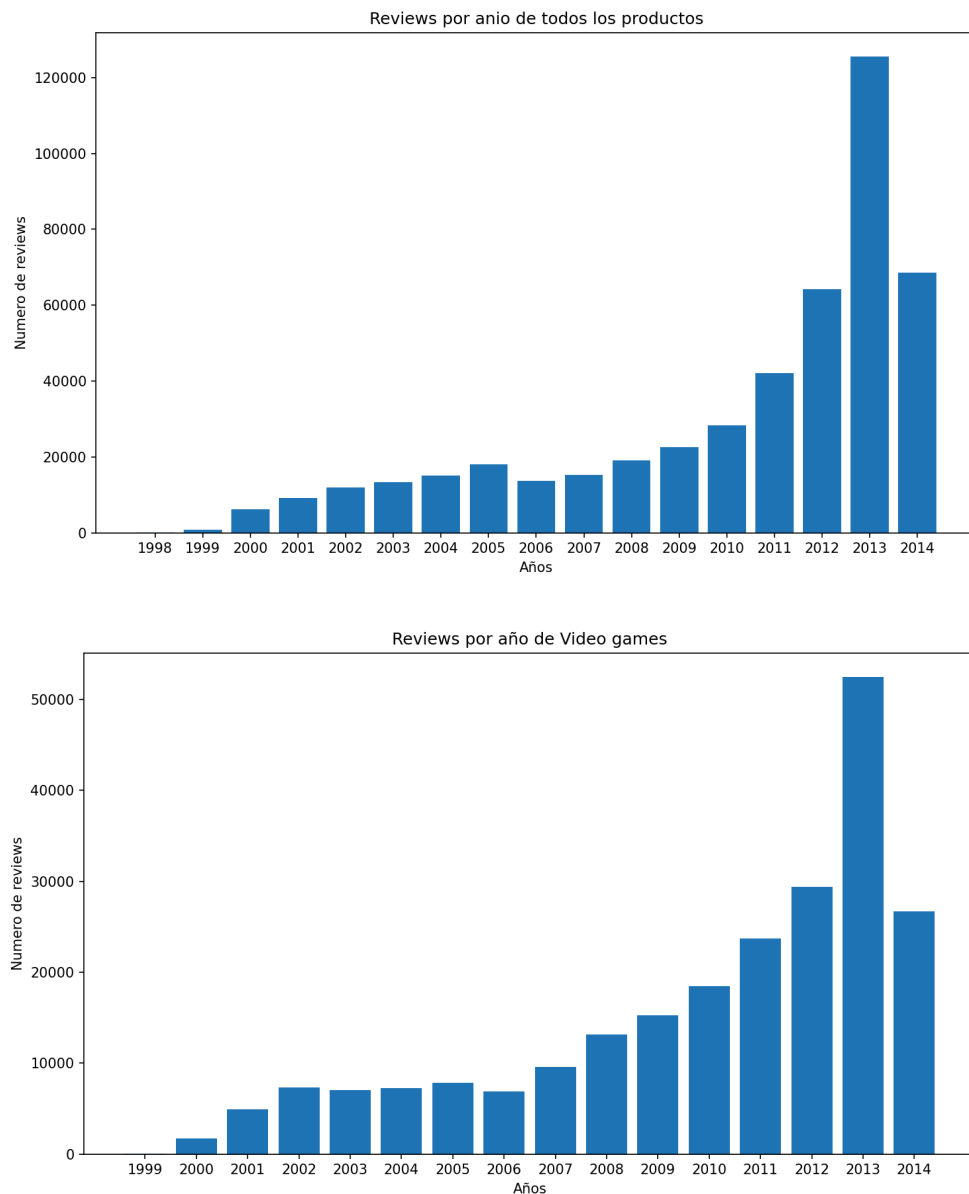


Figura 2.1. Histogramas del número de reviews por año. Ejemplo global para todos los productos y ejemplo restringido a la categoría Video Games.

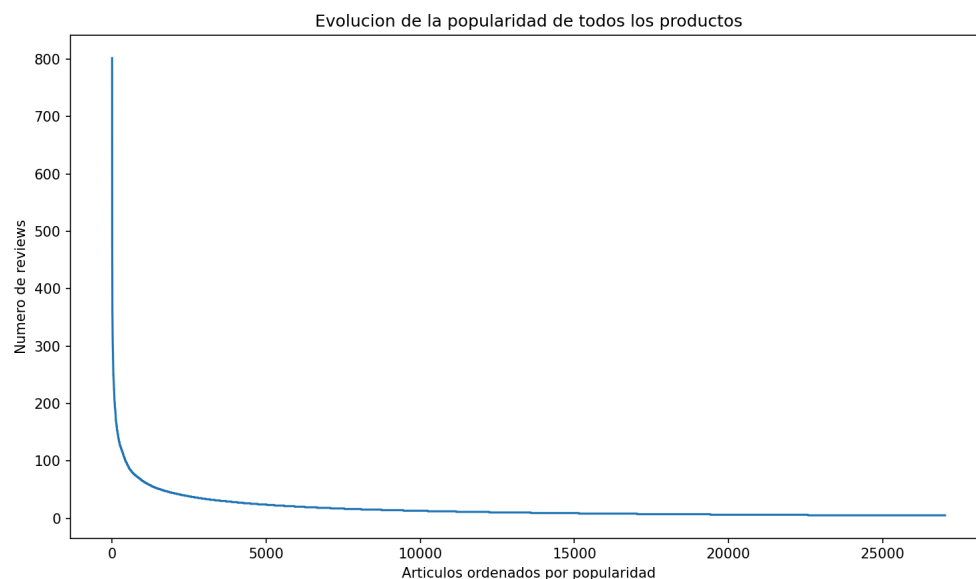
2.5. Evolución de la popularidad de los artículos

La segunda opción muestra una curva con los artículos ordenados de mayor a menor popularidad, entendiendo por popularidad el número de reviews recibidas por cada artículo.

Igual que en la opción anterior, el usuario puede elegir una categoría concreta o todos los productos. Cuando se pide una categoría concreta, el programa vuelve a utilizar REVIEW_CATEGORIA, ya que lo importante no es la categoría del artículo, sino la procedencia de las reviews que se están contando.

En la consulta global, la popularidad se calcula directamente sobre REVIEW, de forma que cada review lógica se cuenta una sola vez, y en la consulta por categoría, la popularidad se obtiene contando únicamente las reviews pertenecientes a la categoría seleccionada.

Después de recuperar los datos, los artículos no se identifican individualmente en el eje x, ya que en esta gráfica no interesa tanto el nombre concreto de cada artículo como la forma general de la distribución. Por eso, en el eje horizontal se representa la posición del artículo dentro del ranking de popularidad, y en el eje vertical su número de reviews. Esta decisión creemos que mejora mucho la legibilidad, especialmente porque el número de artículos es muy grande y etiquetar cada uno de ellos haría la gráfica ilegible.



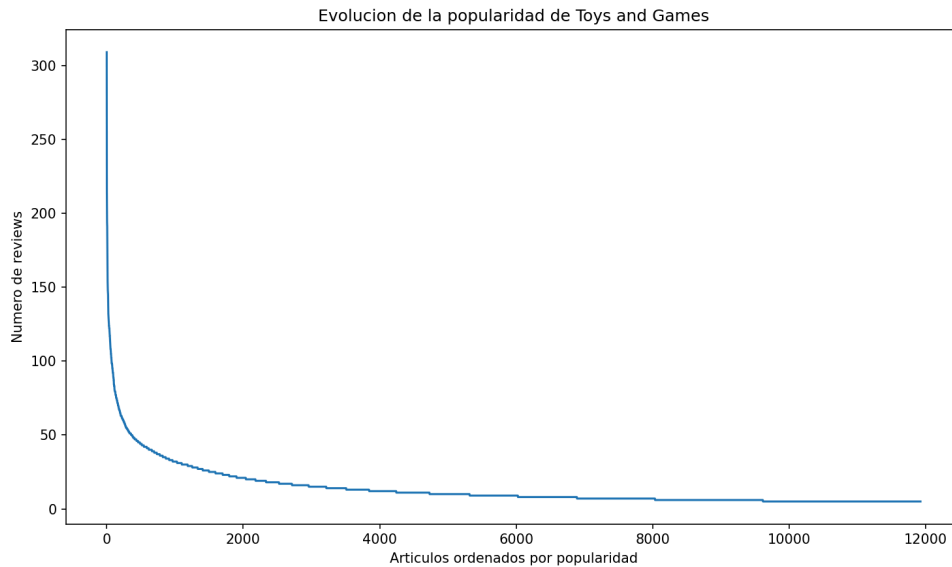


Figura 2.2. Curvas de popularidad de los artículos, ordenados de mayor a menor número de reviews: comparación entre todos los productos y la categoría Toys and Games.

2.6. Histograma por nota

La tercera opción obtiene un histograma del número de reviews para cada valor de overall.

En este caso, el programa permite tres modos de consulta:

- todos los productos
- una categoría concreta
- un artículo individual

La ampliación al caso de artículo individual está permitida expresamente por el enunciado. Por ello, si el usuario escoge ese modo, primero se le pide un *asin* y después se comprueba si ese artículo existe. Si no existe, el programa lo indica por pantalla y no intenta dibujar ninguna gráfica, lo que evita que la aplicación devuelva una gráfica vacía o un error.

Para la visualización se ha decidido fijar explícitamente las cinco notas posibles, de 1 a 5, y construir el histograma sobre esos valores, garantizando así que la gráfica mantenga siempre la misma estructura, aunque en una selección concreta no aparezcan todas las notas.

También aquí, cuando el usuario pide una categoría, la consulta se apoya en `REVIEW_CATEGORIA`. De esa manera, una review solo contribuye al histograma de la categoría en la que realmente apareció.

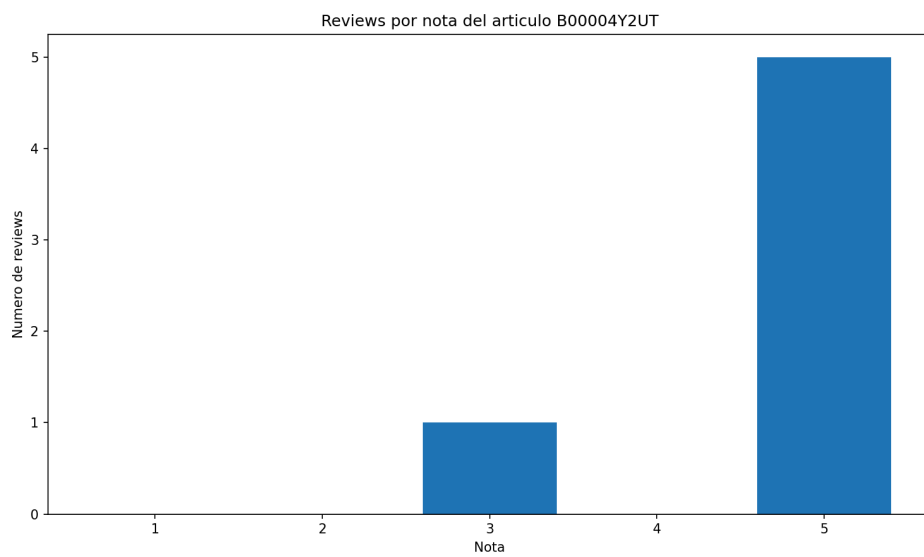
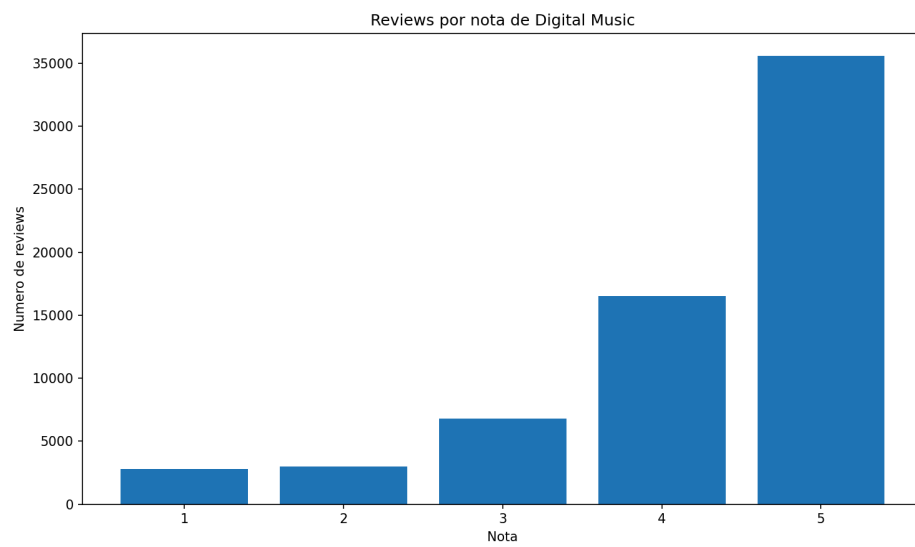


Figura 2.3. Histogramas por nota obtenidos en los tres modos implementados: todos los productos, una categoría concreta (Digital Music) y un artículo individual.

2.7. Evolución de las reviews a lo largo del tiempo

La cuarta opción representa la evolución acumulada del número de reviews a lo largo del tiempo utilizando *unixReviewTime* en el eje x. Para resolverlo de una forma más completa, se han implementado tres modos:

- una sola categoría
- todas juntas
- todas a la vez, separadas por categoría

En el modo de una sola categoría, se recuperan las reviews de la categoría elegida, agrupadas por timestamp, y después en Python se calcula el acumulado, y en el modo global, se consulta directamente la tabla REVIEW sin hacer JOIN con categorías. Esta decisión es importante porque al trabajar sobre REVIEW, cada review lógica se cuenta una sola vez y no se duplica aunque aparezca en más de un dataset. En el modo de todas separadas, la consulta devuelve, para cada categoría y timestamp, cuántas reviews hay, y después el acumulado se construye por separado en Python para dibujar una curva por categoría.

Se ha elegido acumular los valores en Python en lugar de pedir directamente el acumulado a la base de datos porque así el código resulta más sencillo de entender, más portátil y más fácil de adaptar si se quiere cambiar el comportamiento de la gráfica.

Este apartado es especialmente importante porque el modelo actual distingue correctamente entre review lógica global y procedencia categórica. Gracias a ello, la opción de “todas juntas” no sobrecuenta reviews repetidas entre datasets, mientras que las opciones por categoría sí respetan el origen real de cada review.



Figura 2.4. Evolución acumulada del número de reviews a lo largo del tiempo para una sola categoría: ejemplo correspondiente a Musical Instruments.

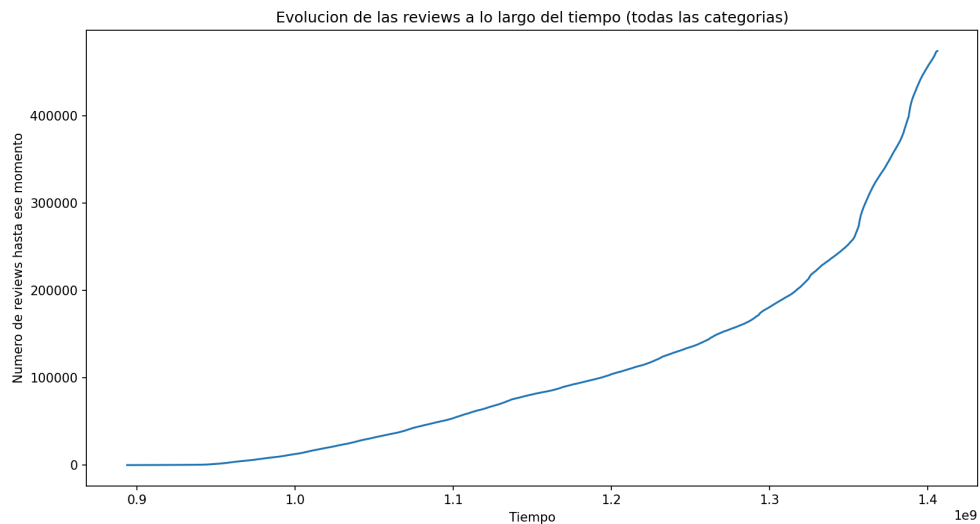


Figura 2.5. Evolución acumulada del número de reviews a lo largo del tiempo para todas las categorías juntas, contando cada review lógica una única vez.

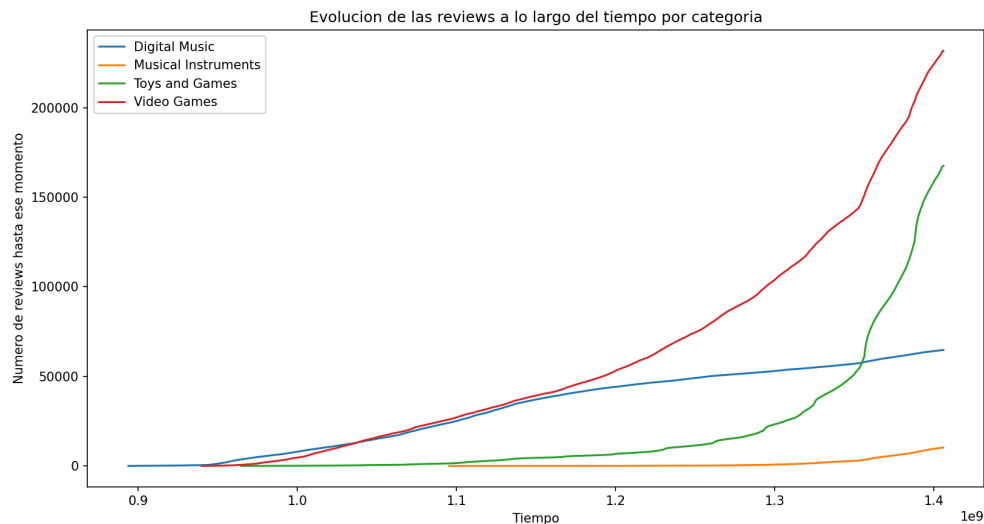


Figura 2.6. Evolución acumulada del número de reviews a lo largo del tiempo separada por categoría, con una curva independiente para cada tipo de producto.

2.8. Histograma de reviews por usuario

La quinta opción representa cuántos usuarios han escrito una determinada cantidad de reviews.

Para construir esta gráfica, primero se calcula para cada usuario cuántas reviews tiene en la tabla REVIEW, y después, a partir de ese resultado intermedio, se vuelve a agrupar por número de reviews para contar cuántos usuarios pertenecen a cada grupo.

No se distingue por categoría, lo cual es coherente con el significado de la gráfica ya que lo que se quiere estudiar aquí es el comportamiento global de participación de los usuarios en el sistema, no su actividad dentro de una sola categoría.

Se ha optado por un histograma en barras de anchura 1 porque el eje x representa un número discreto de reviews. Esa elección hace que la gráfica sea natural y fácil de interpretar.

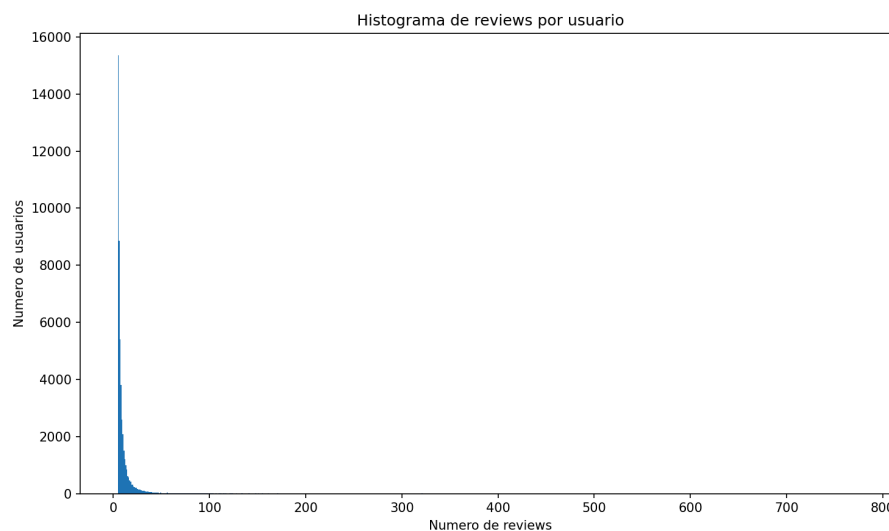


Figura 2.7. Histograma de reviews por usuario, donde el eje x representa el número de reviews realizadas y el eje y el número de usuarios que han escrito esa cantidad.

2.9. Nube de palabras por categoría

La sexta opción genera una nube de palabras utilizando únicamente el campo summary y restringiendo el análisis a una sola categoría.

Esta decisión responde exactamente a lo que pide el enunciado. No se permite generar la nube para todas las categorías juntas, y solo se consideran palabras con longitud mayor que 3 para evitar conectores y artículos.

El procedimiento seguido combina MySQL y MongoDB. Primero, desde MySQL se obtienen los `id_review` de la categoría elegida utilizando `REVIEW_CATEGORIA`, después, con esos identificadores, se consulta MongoDB para recuperar los summaries correspondientes.

Aquí se ha tomado una decisión técnica importante: en lugar de construir una cadena enorme con todos los textos y pasársela directamente a WordCloud, se usa un Counter que va acumulando frecuencias palabra a palabra. Además, la consulta a MongoDB se hace por lotes de tamaño 50000. Esto mejora el rendimiento y evita problemas de memoria o consultas excesivamente grandes, antes de implementar esta mejora, el programa tardaba minutos en devolver la solución. También se ha desactivado `collocations` para evitar combinaciones automáticas de palabras que alteren el recuento simple.

Antes de generar la nube, el programa elimina resúmenes nulos o vacíos. Esta limpieza es necesaria porque esos casos no aportan información y podrían introducir errores.



Figura 2.8. Nube de palabras generada a partir del campo summary para la categoría Video Games, considerando únicamente palabras de longitud mayor que 3.

2.10. Visualización extra

Como visualización adicional distinta de las anteriores, se ha implementado una gráfica de barras con la media de nota por categoría. Se eligió esta opción porque aporta información nueva y no es simplemente una versión modificada de otra gráfica ya incluida. Mientras que el histograma por nota muestra la distribución completa de las puntuaciones, esta visualización resume la valoración media de cada categoría, esto permite comparar de manera inmediata el nivel medio de satisfacción entre tipos de producto.

La consulta vuelve a basarse en REVIEW_CATEGORIA, ya que la media debe calcularse con las reviews realmente asociadas a cada categoría.

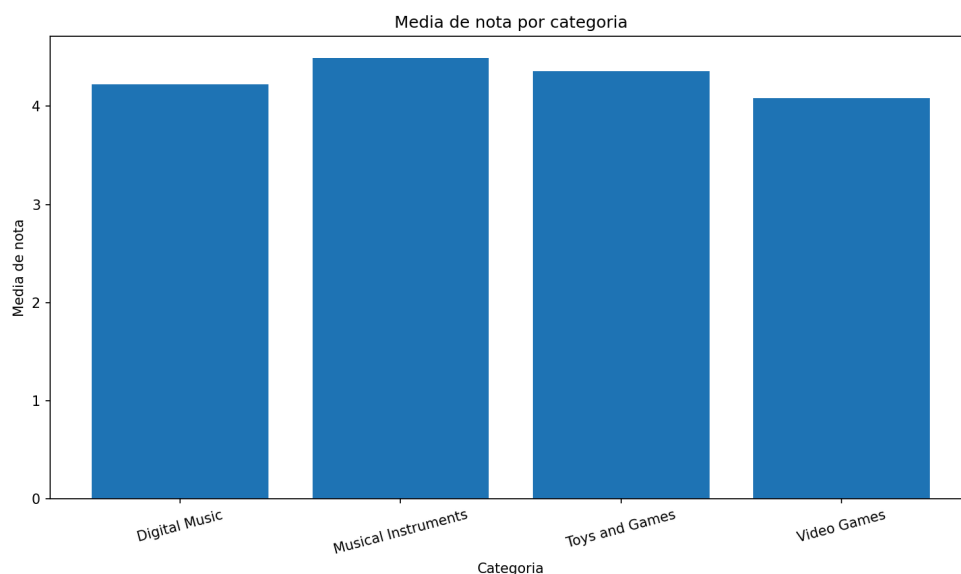


Figura 2.9. Media de la puntuación overall por categoría, calculada a partir de las reviews realmente asociadas a cada tipo de producto mediante REVIEW_CATEGORIA.

2.11. Consideraciones de robustez y usabilidad

Aunque esta parte del proyecto se centra en la visualización, también se ha prestado atención a la robustez del programa.

En primer lugar, el menú se repite hasta que el usuario selecciona la opción de salir, como se pide en el enunciado

En segundo lugar, todas las entradas del usuario se validan, lo que incluye la comprobación de categorías, la verificación de la existencia de un ASIN cuando se consulta un artículo individual y la gestión de opciones no válidas.

En tercer lugar, se han añadido capturas de excepciones para evitar que el programa termine bruscamente si hay algún error durante la ejecución. También se gestiona explícitamente la interrupción manual del usuario mediante teclado.

Por último, las conexiones a MySQL y MongoDB se cierran en el bloque final del programa. Esto evita dejar recursos abiertos innecesariamente pase lo que pase.

3. Tercera parte: aplicación Python junto con Neo4J

3.1. Objetivo de esta tercera parte

En esta tercera parte se ha desarrollado el fichero neo4JProyecto.py, cuyo objetivo es generar distintos grafos en Neo4J a partir de los datos almacenados en MySQL.

La idea general no ha sido trasladar toda la base de datos relacional a Neo4J, sino construir, para cada apartado, únicamente los nodos y relaciones necesarios para visualizar el fenómeno concreto que se quiere estudiar. Esto simplifica cada grafo y hace que la representación sea más clara.

El programa se ha implementado como un menú por terminal con cuatro opciones principales y una opción de salida. Antes de cargar cada nuevo grafo, se eliminan todos los nodos y relaciones previos de Neo4J para evitar mezclar resultados de apartados distintos o de cosas que ya se tuvieran cargadas. Además, el programa muestra un mensaje por pantalla cuando termina la carga, de manera que el usuario sabe cuándo puede consultar el resultado en el navegador de Neo4J.

3.2. Decisiones generales de diseño

En esta parte, MySQL se utiliza como base de cálculo y Neo4J como base de visualización.

Esto significa que las selecciones, filtrados, agregaciones y conteos se realizan primero en MySQL, y después los resultados se transforman en estructuras de Python que finalmente se cargan en Neo4J mediante consultas con UNWIND y MERGE.

Esto tiene varias ventajas. Por un lado, aprovecha que los datos ya están correctamente modelados en MySQL y que ahí es sencillo hacer consultas estructuradas. Por otro, permite mantener Neo4J limpio y centrado en la representación del grafo final, sin necesidad de replicar toda la logica relacional dentro del propio grafo.

Además, se crean restricciones de unicidad en Neo4J para los nodos Usuario, Artículo y TipoArticulo, sobre los campos `id_usuario`, `id_articulo` e `id_categoria` respectivamente. En Neo4J, cada restricción de unicidad genera automáticamente un índice interno, de forma análoga a lo que hace una restricción `UNIQUE` en MySQL, que crea un índice implícito. Gracias a estos índices, las operaciones `MERGE` localizan los nodos existentes de forma eficiente en lugar de recorrer todos los nodos del grafo. Además, las restricciones persisten aunque se eliminen los nodos con `DETACH DELETE`, por lo que solo es necesario crearlas una vez al inicio del programa.

También se ha incorporado un atributo auxiliar llamado `nombre_visible` en los nodos de usuario. Su función es puramente visual: si `reviewerName` existe y no está vacío, se muestra ese valor; si falta o está vacío, se usa `reviewerID`. Esta decisión mejora la legibilidad de los grafos, porque evita que aparezcan usuarios sin etiqueta visible.

3.3. Estructura general del menú Neo4J

El menú del programa tiene las siguientes opciones:

- apartado 4.1: similitudes entre usuarios
- apartado 4.2: usuarios y artículos aleatorios
- apartado 4.3: usuarios y tipos de artículo
- apartado 4.4: artículos populares y artículos en común
- salir

El menú se repite hasta que el usuario decide terminar el programa. Esta estructura sigue directamente la organización pedida por el enunciado, es muy parecida a la que se ha usado en apartados anteriores.

3.4. Apartado 4.1: similitudes entre usuarios

En este apartado, el enunciado pide obtener los 30 usuarios con más reviews, calcular la similitud de Pearson entre cada par de usuarios, almacenar solo las similitudes entre usuarios con artículos en común, cargar ese resultado en Neo4J y mostrar por pantalla qué usuario tiene más vecinos.

La implementación sigue exactamente ese esquema. En primer lugar, se recuperan desde MySQL los usuarios con mayor número de reviews. El número 30 se ha definido como constante, de manera que puede cambiarse fácilmente modificando una sola línea del código.

Después, para esos usuarios se recuperan sus valoraciones por artículo. En la implementación actual, si un mismo usuario tuviera más de una review sobre el mismo artículo, se toma la media de overall para que cada par usuario-artículo quede representado

por una sola valoración. Esta decisión permite trabajar con una estructura limpia para el cálculo de Pearson y evita que un mismo artículo pese varias veces dentro de un mismo usuario. Se preguntó acerca de esto en clase, y se nos comentó que era una buena interpretación.

Además, para el cálculo de r_u y r_v se ha adoptado la interpretación de que ambas medias deben obtenerse a partir de todas las valoraciones realizadas por cada usuario, y no solo de las correspondientes a los artículos que ambos tienen en común. Esta decisión se basa en la propia redacción del enunciado, que define r_u y r_v como las medias de las valoraciones realizadas por los usuarios u y v , respectivamente. Por tanto, se ha entendido que se trata de una media global de cada usuario dentro del conjunto de datos considerado, mientras que el conjunto de artículos que coinciden interviene únicamente en la suma principal de la correlación. Esta elección mantiene una interpretación más fiel de la fórmula y evita que la media de un mismo usuario cambie en función del otro usuario con el que se esté comparando.

A continuación, se construye en Python una estructura de valoraciones por usuario y se calcula la correlación de Pearson para cada par. Solo se almacenan las similitudes entre usuarios que tienen al menos un artículo en común, como indica el enunciado.

Una vez calculadas, las similitudes también se guardan en un fichero auxiliar CSV. El fichero generado se llama `similitudes.csv` y contiene, para cada par de usuarios, sus identificadores, el valor de la correlación de Pearson y el número de artículos en común. Esto se ha hecho porque, facilita la revisión del resultado fuera de Neo4J.

En Neo4J, cada nodo representa un usuario y cada relación bidireccional `SIMILARIDAD` almacena dos propiedades: el valor de Pearson y el número de artículos comunes. Después de la carga, el programa ejecuta una consulta en Neo4J para obtener el usuario con más vecinos y lo muestra por pantalla. Se ha obtenido el siguiente usuario:

Usuario con más vecinos según Neo4J:

```
{'id_usuario': 1792, 'reviewerID': 'A2582KMXLK2P06', 'reviewerName': 'B. E Jackson', 'vecinos': 29}
```

Observación: En Neo4J, todas las relaciones se almacenan internamente con una dirección obligatoria, por lo que el browser siempre las representa con flechas independientemente de cómo se hayan creado. En el caso del grafo de similitudes del apartado 4.1, cada enlace `SIMILARIDAD` representa la correlación de Pearson entre dos usuarios, que es un valor simétrico: $PC(u,v) = PC(v,u)$. Por ello, cada par de usuarios se conecta mediante una única relación, evitando redundancia. Las flechas visibles en el grafo son una representación interna de Neo4J y no implican ninguna direccionalidad. Es decir, las flechas con dirección que se pueden ver en el siguiente grafo, es como si fueran bidireccionales.

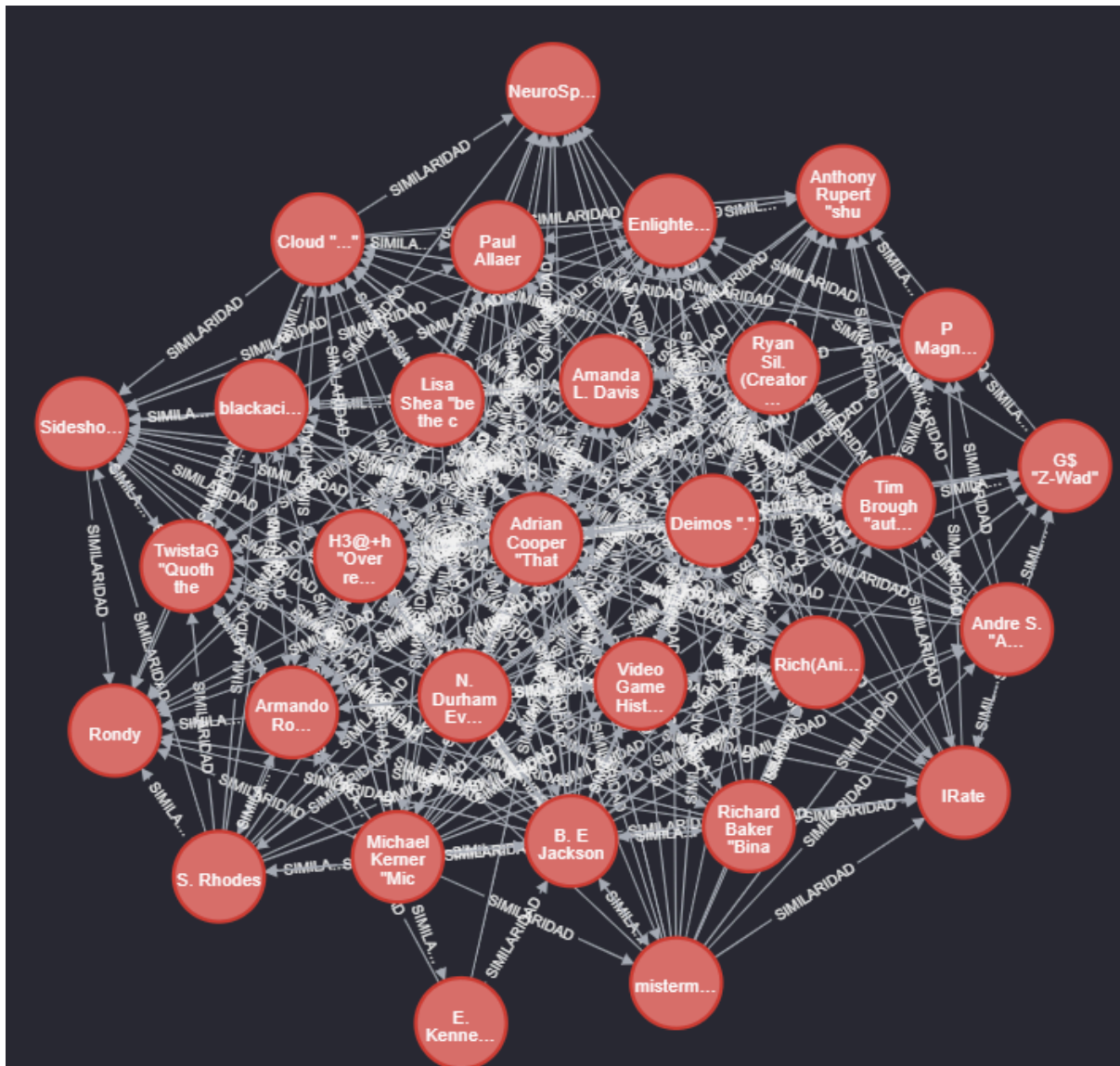


Figura 3.1. Grafo de similitudes entre los 30 usuarios con más reviews, donde cada nodo representa un usuario y cada enlace SIMILARIDAD almacena la correlación de Pearson y el número de artículos en común.

3.5. Apartado 4.2: enlaces entre usuarios y artículos

En este apartado, el enunciado pide seleccionar un número n de artículos aleatorios de un tipo concreto, pedir al usuario tanto la categoría como el valor de n , y mostrar en Neo4J esos artículos junto con todos los usuarios que los han puntuado. La relación entre usuario y artículo debe almacenar la nota y la fecha de la review.

La implementación comienza pidiendo al usuario el nombre exacto de la categoría y el número de artículos aleatorios. La categoría no se valida con una lista fija escrita en Python, sino con la tabla CATEGORIA de MySQL. Esto se ha hecho porque hace que el programa pueda trabajar también con categorías nuevas añadidas más adelante, sin necesidad de modificar el código del menú.

La selección de artículos aleatorios se hace a partir de REVIEW_CATEGORIA. Esto es coherente con el modelo final del proyecto, ya que la categoría correcta no se deduce desde el artículo, sino desde la procedencia de la review.

Además, en la versión final del apartado 4.2 no solo se seleccionan los artículos a partir de esa categoría, sino que también se recuperan únicamente las reviews cuya procedencia pertenece a la misma categoría elegida, lo cual era necesaria porque, si un artículo aparece en más de un dataset, recuperar todas sus reviews sin filtrar volvería a introducir el error semántico que ya se había detectado en la primera parte del proyecto. Gracias a este cambio, el grafo de Neo4J muestra solo las valoraciones realmente pertenecientes a la categoría seleccionada.

En Neo4J, el resultado se representa mediante nodos Usuario, nodos Artículo y relaciones PUNTUO. En cada relación se almacenan overall, unixReviewTime y reviewTime.

Justificación del diseño del apartado 4.2. La decisión crítica de este apartado es que las reviews mostradas en Neo4J no se obtienen con un simple SELECT sobre REVIEW filtrando por id_articulo, sino que se recuperan mediante un JOIN con REVIEW_CATEGORIA y CATEGORIA para hacer que la procedencia categórica de cada review coincida con la categoría elegida por el usuario. Este diseño es consecuencia directa del modelo adoptado en la primera parte del proyecto, en la que un mismo artículo puede aparecer en varios datasets, por lo que una review lógica asociada a ese artículo puede haber sido publicada en una categoría distinta a la que se está visualizando. Si se recuperaran todas las reviews del artículo sin filtrar, se reintroduciría el mismo error que se corrigió al sustituir ARTICULO_CATEGORIA por REVIEW_CATEGORIA en la primera parte, donde los usuarios aparecerían conectados al artículo con valoraciones que en realidad pertenecen a otra categoría. Filtrando por REVIEW_CATEGORIA se garantiza que el grafo mostrado representa exclusivamente la actividad real de los usuarios dentro de la categoría seleccionada, lo que mantiene la coherencia del modelo entre MySQL y Neo4J. Por esto mismo, la selección aleatoria de los n artículos también se hace a partir de REVIEW_CATEGORIA y no desde ARTICULO, garantizando que los artículos elegidos son realmente representativos de la categoría pedida, y no artículos que solo aparecen en ella por pertenecer a varios datasets.

Ejemplo para categoría = Digital Music y número de artículos aleatorios = 5

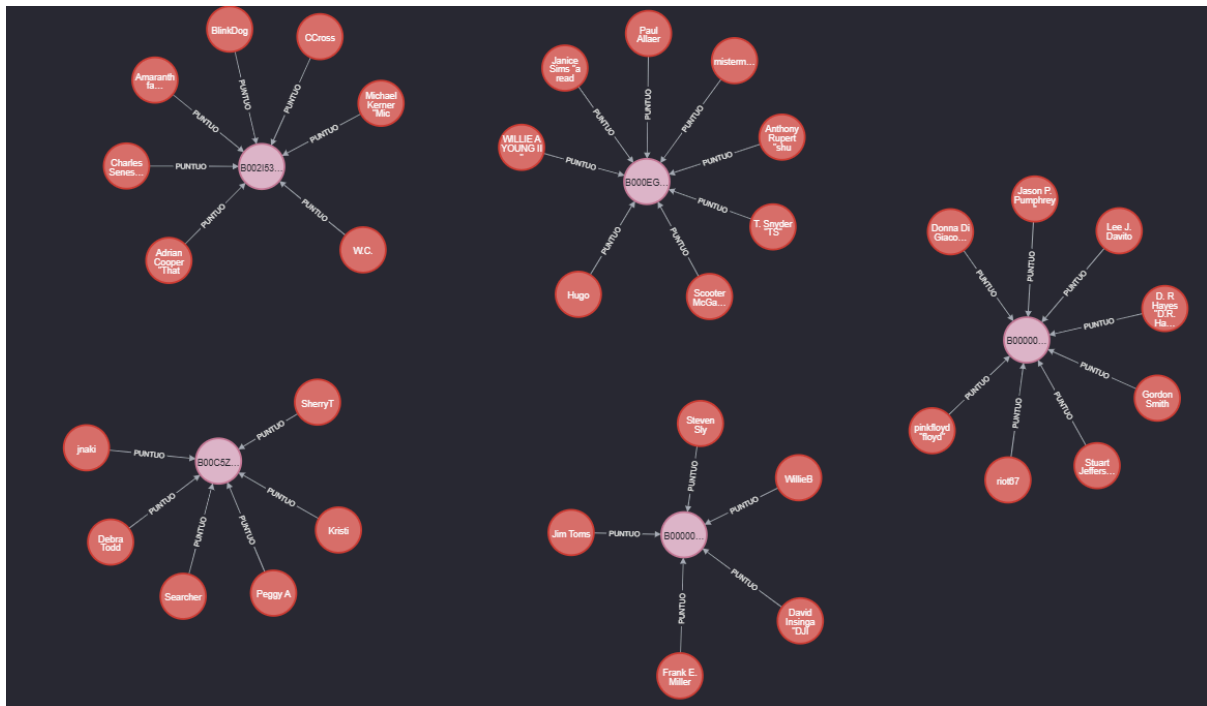


Figura 3.2. Grafo del apartado 4.2 en Neo4J para la categoría Digital Music y 5 artículos aleatorios, mostrando los artículos seleccionados, los usuarios que los han puntuado y las relaciones PUNTUO con la nota y el instante temporal de la review.

3.6. Apartado 4.3: usuarios que han puntuado más de un tipo de artículo

En este apartado, el enunciado pide seleccionar los primeros 400 usuarios ordenados por nombre y, entre ellos, quedarse con aquellos que han puntuado artículos de más de un tipo. En Neo4J deben mostrarse nodos de usuario y nodos de tipo de artículo, no artículos concretos, y las relaciones deben indicar cuántos artículos ha consumido el usuario de ese tipo.

La implementación comienza seleccionando los primeros 400 usuarios ordenados por reviewerName. Además, solo se incluyen usuarios cuyo reviewerName no es nulo ni vacío. Esto se hace ya que por lo que hemos investigado, con en el orden ASCII que utiliza MySQL, tanto NULL como la cadena vacía se colocan al principio del ranking, por lo que sin este filtro ocuparían posiciones dentro de los 400 primeros y desplazarían a usuarios con nombre real. El filtro garantiza que los 400 seleccionados son efectivamente los 400 primeros usuarios con nombre, que es lo que el enunciado pide al hablar de ordenación alfabética. Este filtro es también la razón de ser del índice manual idx_usuario_reviewer_name descrito en el apartado 1.5.2.

Después, para esos usuarios se calcula cuántos artículos distintos han puntuado en cada categoría. Aquí se cuenta COUNT(DISTINCT r.id_articulo) y no el número total de reviews, porque el enunciado habla del número de artículos consumidos de ese tipo, no del número bruto de valoraciones.

Además, se ha añadido una comprobación adicional para evitar falsos positivos. Un usuario no debe considerarse multicategoría simplemente por haber puntuado un único artículo que aparece en más de una categoría. Por eso, además del número de categorías distintas, se calcula también el número total de artículos distintos puntuados por cada usuario, y solo se conserva a aquellos que tienen más de una categoría y además más de un artículo distinto en total.

En Neo4J, el resultado se representa con nodos Usuario, nodos TipoArticulo y relaciones CONSUMIO_TIPO, donde la propiedad numero_articulos indica cuántos artículos distintos ha puntuado ese usuario dentro de esa categoría.

Esta parte que se explica a continuación es muy importante

Una decisión de diseño muy importante en este apartado, que no es obvia a simple vista, es que el tipo de artículo de cada usuario se obtiene desde REVIEW_CATEGORIA y no deduciéndolo desde ARTICULO. Esto es consecuencia directa de la corrección explicada en el apartado 1.3.3, la categoría pertenece conceptualmente a la procedencia de la review, no al artículo, porque un mismo asin puede aparecer en varios datasets. Si en este apartado la categoría se dedujese desde el artículo, un usuario que hubiera valorado únicamente un artículo presente en varios datasets aparecería como multicategoría sin serlo realmente, y el filtro $\text{total_articulos} > 1$ tampoco lo descartaría. Por tanto, el grafo resultante cuenta como multicategoría solo a los usuarios que realmente han participado en más de un dataset con artículos distintos, lo cual según nuestra interpretación, es el comportamiento que el enunciado busca y el más lógico. Conviene mencionar que bajo un modelo alternativo, donde la categoría se almacenase a nivel de artículo duplicando el artículo por cada categoría a la que pertenece, el número de usuarios resultante sería mayor, pero incluiría falsos multicategoría generados por la propia estructura del modelo, podría causar que usuarios que solo aparezcan en un json de reviews original, aparezcan en el grafo.

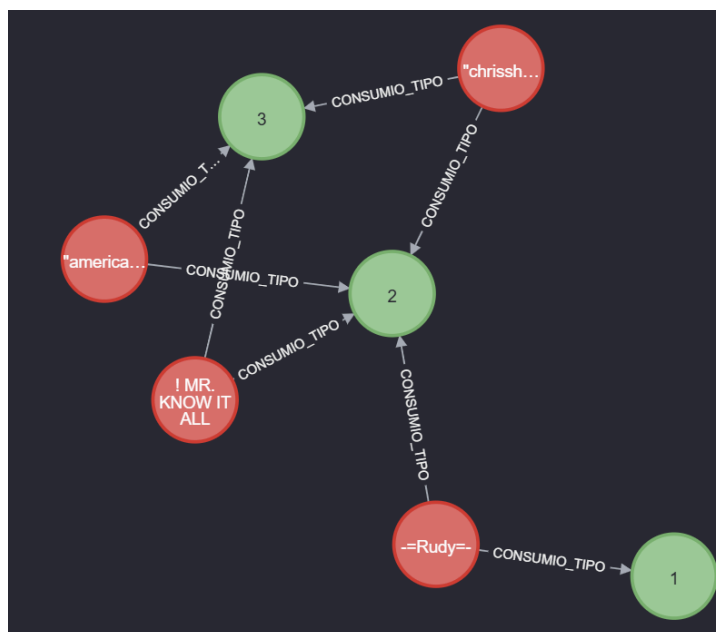


Figura 3.3. Grafo de usuarios que han puntuado más de un tipo de artículo.

3.7. Apartado 4.4: artículos populares y artículos en común entre usuarios

En este apartado, el enunciado pide seleccionar los 5 artículos más populares con menos de 40 reviews, mostrarlos en Neo4J junto con todos los usuarios que los han puntuado y, además, crear enlaces entre usuarios indicando cuántos de esos artículos han puntuado en común.

La implementación utiliza dos constantes para recoger exactamente esas condiciones: el número de artículos populares a recuperar y el máximo de reviews permitido por artículo. Esto hace que el comportamiento del programa sea fácil de ajustar.

Primero, desde MySQL se recuperan los artículos que cumplen la condición. Después se obtienen todas las reviews asociadas a esos artículos y, por otro lado, se calcula cuántos artículos de ese conjunto tiene en común cada par de usuarios.

En Neo4J se crean dos tipos de relaciones. La primera es PUNTUO, entre Usuario y Artículo, con las propiedades overall, unixReviewTime y reviewTime. La segunda es ARTICULOS_EN_COMUN, entre Usuario y Usuario, con una propiedad articulos_comunes que indica cuántos artículos del conjunto seleccionado han puntuado ambos usuarios.

Este apartado no necesita un filtrado por categoría porque el enunciado no lo pide. Aquí lo que se estudia es la popularidad global de ciertos artículos y la intersección entre los usuarios que han interactuado con ellos.

Observación: esta foto se muestra con fondo blanco pues al haber tantos nodos se observa mejor.

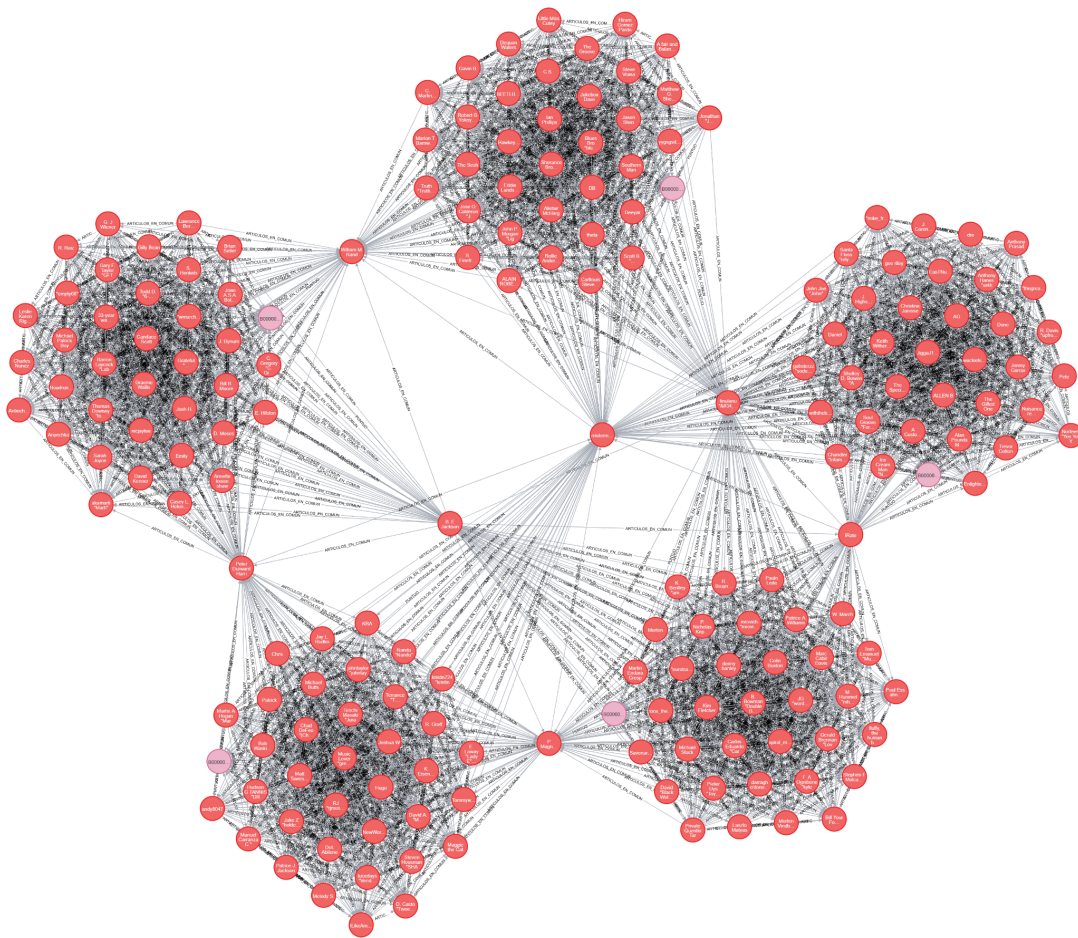


Figura 3.4. Grafo de los 5 artículos más populares con menos de 40 reviews.

3.8. Consideraciones comunes sobre limpieza y carga en Neo4J

En los cuatro apartados de esta tercera parte se ha decidido vaciar Neo4J antes de cargar el nuevo grafo. Esta decisión evita mezclar visualizaciones de apartados distintos y hace que cada ejecución se pueda interpretar de forma aislada. Como se ha explicado anteriormente.

También en todos los casos el programa imprime un mensaje de finalización de carga y una consulta recomendada del tipo `MATCH (n) RETURN n` para facilitar que el usuario compruebe rápidamente el resultado en el navegador de Neo4J.

Otra decisión común ha sido no cargar información innecesaria. Cada apartado construye únicamente el grafo mínimo necesario para representar el fenómeno pedido. Esto hace que la visualización sea más clara y que Neo4J no se llene con nodos o relaciones que no aportan nada a ese apartado concreto.

4. Cuarta parte: nuevos datos

4.1. Objetivo de esta cuarta parte

En esta cuarta parte, el enunciado pide seleccionar un fichero adicional distinto de los cuatro iniciales y diseñar un mecanismo para insertarlo en la base de datos ya construida.

Para resolverlo se ha implementado el fichero `inserta_dataset.py`. En nuestro caso, el dataset adicional que hemos elegido es Office Products, la idea principal de esta parte ha sido que la inserción del nuevo dataset no obligue a rehacer toda la carga desde cero ni a modificar la lógica del proyecto.

4.2. Decisión general de diseño

La decisión fundamental en esta parte ha sido reutilizar el mismo modelo de carga definido en `load_data.py`.

Esto significa que el nuevo dataset no se trata como una excepción ni como una estructura separada, sino que se integra exactamente con la misma lógica que ya se utilizó para los cuatro datasets originales. Se ha tomado esta decisión ya que es la más razonable porque garantiza consistencia entre la carga inicial y la inserción posterior.

Además, este enfoque respeta el hecho de que un mismo usuario puede aparecer en varios datasets y de que una misma review lógica puede estar repetida en distintos ficheros. Por tanto, se incorporan sus datos al modelo ya creado.

4.3. Configuración del nuevo dataset

En `inserta_dataset.py` se han definido dos constantes para indicar el nombre de la nueva categoría y la ruta del fichero JSON que se quiere insertar.

La ventaja de esta solución es que permite cambiar de dataset nuevo modificando únicamente esas constantes, sin necesidad de tocar el resto del código, con esto, el `.py` no queda ligado exclusivamente a Office Products, sino que puede reutilizarse para cualquier otro dataset adicional con la misma estructura.

4.4. Inserción de la nueva categoría

Antes de insertar las reviews del nuevo fichero, el programa comprueba si la categoría ya existe en la tabla CATEGORIA.

Si ya existe, reutiliza su `id_categoria`. Si no existe, la inserta y obtiene el nuevo identificador. Esta decisión evita duplicar categorías y, al mismo tiempo, permite ampliar el conjunto de tipos de producto sin modificar manualmente la base de datos, además elimina posibles errores en caso de que se ejecute varias veces.

4.5. Integración con el modelo relacional y documental

La inserción del nuevo dataset sigue exactamente la misma lógica que la carga principal.

Para cada línea del JSON, el programa: lee la review línea a línea, sin cargar el fichero completo en memoria, valida que existan los campos imprescindibles reviewerID, asin y unixReviewTime, convierte reviewTime al formato de fecha utilizado por MySQL, obtiene o reutiliza el id interno del usuario, obtiene o reutiliza el id interno del artículo y comprueba si ya existe una review lógica con la tripleta id_usuario, id_articulo y unixReviewTime

Si la review no existe todavía, se inserta en REVIEW, se registra su procedencia en REVIEW_CATEGORIA y se añade su parte documental al buffer que después se enviará a MongoDB.

Si la review ya existía, no se duplica ni en REVIEW ni en MongoDB. En ese caso, únicamente se añade la nueva relación en REVIEW_CATEGORIA para dejar constancia de que esa review también apareció en el nuevo dataset.

Esta decisión es esencial para mantener la coherencia del modelo. Gracias a ella, la base de datos conserva una sola vez cada review lógica, pero sigue registrando en qué categorías o datasets apareció.

4.6. Reutilización de usuarios y artículos

Una parte importante del diseño consiste en no volver a insertar usuarios ni artículos que ya existían.

Para ello, inserta_dataset.py reutiliza las funciones auxiliares de load_data.py que obtienen el identificador interno del usuario y del artículo. Si el reviewerID o el ASIN ya estaban presentes en la base de datos, se recupera su identificador existente. Si no estaban, se insertan.

Esto evita duplicidades y mantiene el modelo uniforme. También es coherente con el enunciado, que ya advertía de que un mismo usuario puede haber realizado reviews en distintos tipos de productos.

4.7. Uso de cachés y procesamiento eficiente

Al igual que en load_data.py, en inserta_dataset.py se utilizan cachés en memoria para usuarios y artículos, con el objetivo de reducir el número de llamadas a los servidores.

Estas cachés se inicializan vacías y se van rellenando solo con los identificadores que van apareciendo en el nuevo dataset.

El fichero también se procesa línea a línea. Esta decisión no es solo una mejora de eficiencia, sino una exigencia metodológica coherente con el planteamiento general del proyecto, el cual es no cargar ficheros completos en memoria

4.8. Sincronización con MongoDB

La parte documental de las nuevas reviews se inserta en MongoDB por lotes, reutilizando también la misma lógica de `load_data.py`.

Esta decisión mantiene el rendimiento y, además, conserva la sincronización entre MySQL y MongoDB, además, si ocurre un fallo parcial o total al insertar un lote en MongoDB, la lógica auxiliar utilizada elimina de MySQL las filas correspondientes del lote afectado, de forma que no queden reviews estructuradas sin su parte documental asociada. Básicamente igual que se hacía antes.

4.9. Tratamiento de errores y limpieza del proceso

Durante la inserción, si una review concreta produce un error al insertarse en MySQL, esa review se descarta y el programa continúa con la siguiente. Se ha preferido este comportamiento frente a cancelar toda la carga, porque en ficheros grandes puede ocurrir que una línea aislada dé algún problema, y no tendría sentido perder por ello toda la inserción del dataset.

5. Quinta parte: Más visualización y relación con Machine learning.

5.1. Uso de PowerBI como alternativa de visualización

En este apartado se han recreado en Power BI cuatro de las visualizaciones implementadas en Python, con el objetivo de ofrecer una representación alternativa.

Para obtener los datos, se ejecutaron en MySQL Workbench las mismas consultas SQL que utiliza el programa `menu_visualizacion.py`, y los resultados se exportaron a ficheros CSV, llevándolos a Power BI.

Las cuatro visualizaciones recreadas son las siguientes (aunque las explicaciones de los gráficos son muy similares a las ya hechas en el apartado sobre `menu_visualizacion.py`):

- **Reviews por año de todos los productos**. Se trata de un gráfico de barras que muestra el número de reviews publicadas cada año, agrupando por el campo `reviewTime`. La consulta utilizada, agrupa directamente sobre la tabla `REVIEW`, sin filtrar por categoría, de modo que cada review lógica se cuenta una sola vez. El resultado es idéntico al que produce el histograma de la opción 1 del menú de visualización en Python cuando se selecciona "Todo".
- **Número de reviews por nota**. Histograma que representa cuántas reviews se han registrado para cada puntuación overall (de 1 a 5) considerando todos los productos. En Power Query se configuró la columna de nota como tipo texto para que Power BI

la tratara como un eje categórico y no intentara resumir ni interpolar los valores. De este modo, el eje X muestra las cinco notas individuales, igual a como aparecen en la gráfica generada por Python.

- **Histograma de reviews por usuario.** Muestra en el eje X el número de reviews escritas y en el eje Y cuántos usuarios han escrito esa cantidad. La distribución resultante es muy asimétrica, lo que quiere decir que la gran mayoría de usuarios han escrito muy pocas reviews, mientras que unos pocos usuarios alcanzan centenas de valoraciones. Esta gráfica no distingue por tipo de producto, tal y como establece el enunciado.
- **Nota media por categoría.** Gráfico de barras que compara la puntuación media (overall) de las reviews asociadas a cada categoría. La consulta se basa en REVIEW_CATEGORIA para calcular la media únicamente con las reviews que realmente pertenecen a cada tipo de producto, de forma coherente con el modelo de datos adoptado en la primera parte del proyecto. Las cuatro categorías presentan medias cercanas entre sí, todas por encima de 4, lo que indica un nivel general de satisfacción alto en todos los tipos de producto.

En la Figura 5.1 se muestra el panel completo de Power BI con las cuatro visualizaciones dispuestas en un único dashboard.

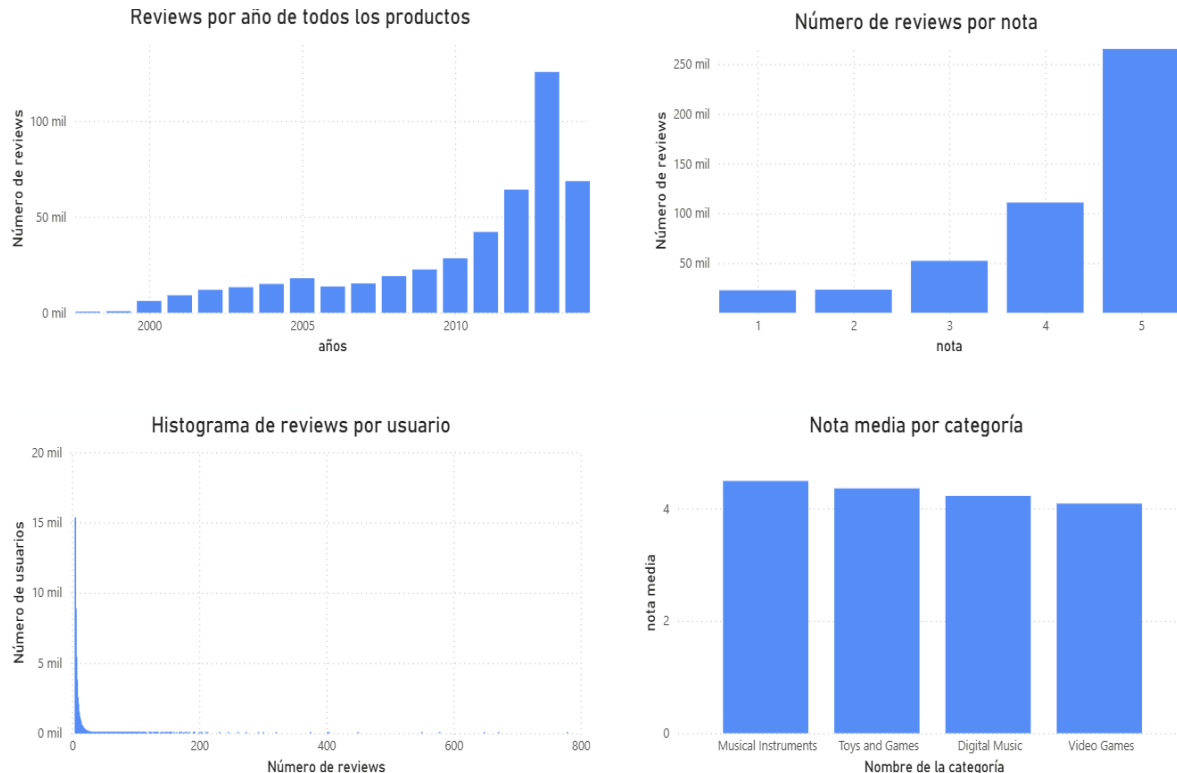


Figura 5.1. Panel completo de Power BI.

5.2. Propuesta de modelo de Machine Learning para recomendación

5.2.1. Planteamiento del problema

El objetivo es recomendar artículos a un usuario que todavía no los ha consumido. Es decir, dado un usuario u y un tipo de producto, el sistema debe sugerir artículos que ese usuario probablemente valoraría de forma positiva, basándose en el comportamiento pasado de todos los usuarios almacenados en la base de datos.

Para resolverlo se propone un enfoque de filtrado colaborativo basado en similitud entre usuarios (user-based collaborative filtering). La idea central es sencilla: si dos usuarios han puntuado de forma parecida los mismos artículos en el pasado, es probable que también coincidan en sus preferencias sobre artículos que solo uno de ellos ha consumido. Este enfoque no necesita conocer nada sobre el contenido del artículo ni sobre los textos de las reviews; se basa exclusivamente en los patrones de puntuación.

5.2.2. Datos utilizados

Del modelo de datos ya implementado se aprovechan los siguientes aspectos:

1. De MySQL se obtiene la tabla REVIEW, que contiene las valoraciones (overall) de cada usuario sobre cada artículo, identificados mediante `id_usuario` e `id_articulo`. También se utiliza REVIEW_CATEGORIA y CATEGORIA para poder restringir las recomendaciones a un tipo de producto concreto si se desea.
2. El campo overall (puntuación de 1 a 5) es el dato principal del modelo, ya que representa la preferencia explícita del usuario.
3. No es necesario acceder a MongoDB para este modelo, ya que los campos textuales (summary, reviewText) no intervienen en el filtrado colaborativo.

5.2.3. Fases del modelo

Fase 1: Construcción de la matriz usuario-artículo.

Se construye una estructura (por ejemplo, un diccionario de diccionarios) donde cada fila representa un usuario y cada columna un artículo. El valor de cada celda es la puntuación overall que ese usuario dio a ese artículo. La mayoría de las celdas estarán vacías, ya que cada usuario solo ha puntuado una pequeña fracción de todos los artículos disponibles. Si un usuario ha valorado el mismo artículo más de una vez, se utiliza la media de sus puntuaciones, de igual forma con la decisión adoptada en el apartado 4.1 del proyecto.

Fase 2: Cálculo de similitudes entre usuarios.

Para cada par de usuarios que comparten al menos un artículo puntuado, se calcula una medida de similitud. En coherencia con el resto del proyecto, se usa la correlación de

Pearson, pues ya se ha implementado en el apartado 4.1. que tiene la ventaja de ser independiente de si un usuario tiende a puntuar más alto o más bajo que otro en general.

Para predecir qué nota daría un usuario a un artículo, una primera idea sería mirar a todos los usuarios que han valorado ese artículo y combinar sus notas según lo parecidos que sean al usuario objetivo. El problema es que esto introduce ruido y aumenta mucho su coste computacional, lo que empeora el resultado. Por eso, aunque sí calculamos la correlación de Pearson entre el usuario objetivo y el resto de usuarios, a la hora de predecir no las usamos todas: nos quedamos solo con los k usuarios más parecidos (en nuestro caso k = 20, los que tienen mayor correlación positiva). Así la predicción se apoya únicamente en usuarios con gustos realmente similares, lo que hace el cálculo más rápido. El valor de k se puede ajustar por validación cruzada, como se vio en aprendizaje automático, pero en nuestro caso lo hemos ajustado de forma experimental.

Fase 3: Predicción de puntuaciones.

Para predecir la puntuación que el usuario u daría a un artículo “i” que no ha consumido, se utiliza una media ponderada de las puntuaciones que sus vecinos más similares han dado a ese artículo. La fórmula es la siguiente:

$$pred(u, i) = \bar{r}_u + \frac{\sum_{v \in V} sim(u, v) * (r_{vi} - \bar{r}_v)}{\sum_{v \in V} |sim(u, v)|}$$

Donde:

- $pred(u, i)$: puntuación predicha del usuario u para el artículo i
- $\bar{r}_u$: media de todas las puntuaciones del usuario u
- $\bar{r}_v$: media de todas las puntuaciones del vecino v
- r_{vi} : puntuación que el vecino v dio al artículo i
- $sim(u, v)$: correlación de Pearson entre u y v
- V: conjunto de vecinos de u que han puntuado el artículo i

Esta fórmula tiene una interpretación clara, se parte de la media del usuario objetivo y se ajusta hacia arriba o hacia abajo según si los vecinos similares puntuaron el artículo por encima o por debajo de su propia media.

Fase 4: Generación de recomendaciones.

Una vez se puede predecir la puntuación para cualquier artículo no consumido, el proceso de recomendación consiste en:

1. Identificar todos los artículos que el usuario u no ha puntuado (opcionalmente, restringidos a una categoría concreta mediante REVIEW_CATEGORIA).
2. Calcular la predicción de puntuación para cada uno de ellos.
3. Ordenar los artículos de mayor a menor predicción.
4. Devolver los N artículos con mayor predicción estimada (por ejemplo, los 10 primeros).

5.2.4. Evaluación del modelo

Para comprobar si el modelo funciona correctamente antes de ponerlo en producción, se puede utilizar una estrategia de validación sencilla. Se divide el conjunto de puntuaciones conocidas de cada usuario en dos partes: un conjunto de entrenamiento (por ejemplo, el 80 % de las reviews) y un conjunto de test (el 20 % restante). El modelo se entrena únicamente

con las puntuaciones de entrenamiento y después intenta predecir las puntuaciones del conjunto de test. La calidad de las predicciones se mide con el error absoluto medio (MAE), que indica cuántos puntos se desvía de media la predicción respecto a la puntuación real.

5.2.5. Diagrama del modelo

El Diagrama 1 resume el flujo completo del sistema de recomendación, desde la obtención de los datos hasta la lista final de artículos recomendados.

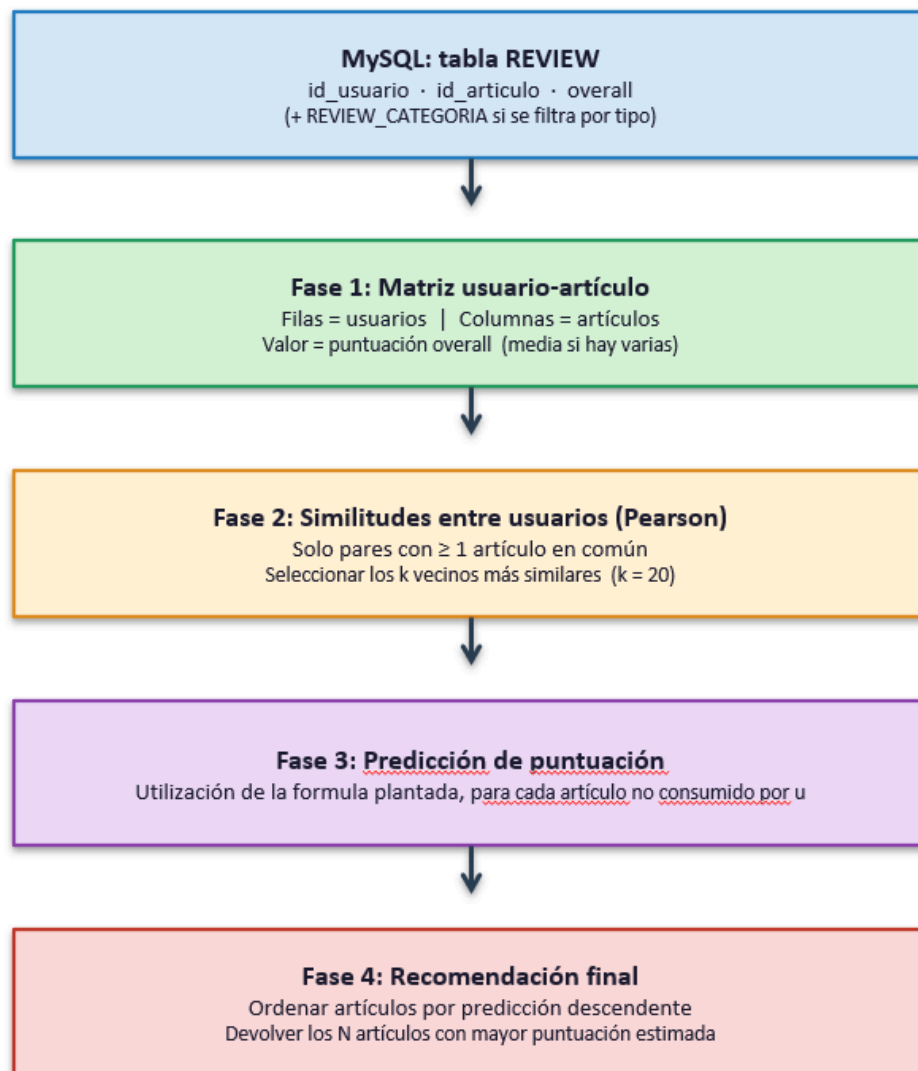


Diagrama 1. Sistema de recomendación basado en filtrado colaborativo por similitud entre usuarios

Observación: Si se desea más información sobre este apartado, se recomienda encarecidamente visitar la sección del primer opcional

5.3. Implementación del sistema de recomendación por popularidad

Para el tercer punto de la quinta parte se ha desarrollado el fichero `recomendacion.py`, un módulo independiente que implementa un sistema de recomendación sencillo basado en popularidad.

Dado el identificador original de un usuario (`reviewerID`) y una categoría de producto, el programa devuelve los 10 artículos más populares de esa categoría que el usuario aún no ha consumido. La popularidad se define como el número de reviews que tiene cada artículo dentro de la categoría seleccionada, utilizando la tabla `REVIEW_CATEGORIA` como fuente de procedencia categórica, de forma coherente con el resto del proyecto.

La consulta SQL principal realiza un JOIN entre `REVIEW`, `REVIEW_CATEGORIA`, `CATEGORIA` y `ARTICULO`, filtra por la categoría elegida y excluye mediante una subconsulta `NOT IN` todos los artículos que el usuario ya ha valorado, independientemente de la categoría en la que los haya valorado. Los resultados se agrupan por artículo, se ordenan por número de reviews en orden descendente y se limitan a los 10 primeros. De esta forma, se recomiendan los artículos con mayor actividad de la comunidad que el usuario todavía no conoce.

El módulo incluye un menú interactivo que permite repetir consultas para distintos usuarios y categorías sin necesidad de reiniciar el programa. Antes de solicitar la categoría, se muestra la lista de categorías disponibles mediante la función `obtener_categorias`, y la validación de la entrada se realiza por comparación directa con dicha lista. También se comprueba que el `reviewerID` introducido exista en la base de datos antes de ejecutar la consulta de recomendación.

Los parámetros de conexión a MySQL se importan directamente desde `configuracion.py` mediante la sentencia `from configuracion import ...`, manteniendo el mismo estilo de importación que el resto de ficheros del proyecto.

6. Opcionales

6.1. Primer Opcional: implementación del modelo de Machine Learning

Se ha implementado el modelo de filtrado colaborativo propuesto en la sección 5.2 del informe en un fichero independiente llamado `modelo_ml.py`. El programa pone en práctica las cuatro fases descritas teóricamente: construcción de la matriz usuario-artículo, cálculo de similitudes mediante correlación de Pearson, predicción de puntuaciones y generación de recomendaciones.

Al iniciar, el programa carga todas las valoraciones de la tabla `REVIEW` y las divide en un conjunto de entrenamiento (80%) y un conjunto de test (20%). Los usuarios con menos de cinco valoraciones se asignan íntegramente al entrenamiento, ya que no disponen de suficientes datos para evaluar predicciones de forma significativa. La división se realiza con una semilla fija para garantizar la reproducibilidad de los resultados.

El cálculo de Pearson reutiliza exactamente la misma interpretación adoptada en el apartado 4.1 del proyecto..

El programa ofrece dos funcionalidades a través de un menú por terminal:

- Evaluación del modelo. Para cada valoración del conjunto de test, se buscan los $k = 20$ vecinos más similares del usuario (con Pearson positiva) y se predice la puntuación aplicando la fórmula de media ponderada descrita en la sección 5.2.3. La calidad del modelo se mide con el MAE (error absoluto medio), que indica cuántos puntos se desvía, de media, la predicción respecto a la puntuación real. Si el MAE resulta cercano a 1, significa que el modelo se equivoca de media en aproximadamente un punto sobre una escala de 1 a 5, lo cual es un resultado razonable para este tipo de enfoque.
- Recomendaciones interactivas. Dado un reviewerID y una categoría (o todas), el programa busca los vecinos del usuario, predice la puntuación para los artículos que no ha consumido y muestra los 10 con mayor predicción estimada. Esto permite comprobar de forma directa que el modelo genera recomendaciones coherentes.

Observación: la evaluación del modelo y las recomendaciones interactivas trabajan de forma distinta con los conjuntos de datos. En la evaluación (opción 1) se usan tanto los vecinos y las predicciones se calculan a partir del entrenamiento, pero esas predicciones se comparan con las valoraciones reales del conjunto de test para obtener el MAE. En cambio, en las recomendaciones interactivas (opción 2) solo interviene el entrenamiento, los vecinos se buscan en él, los artículos candidatos son los que el usuario no ha valorado en entrenamiento y las predicciones se apoyan en las valoraciones de entrenamiento de los vecinos. De este modo se mantiene la coherencia con el planteamiento de Machine Learning del proyecto, ya que el conjunto de test cumple su función natural (medir el error del modelo sobre datos no vistos) sin contaminar las recomendaciones generadas para un usuario concreto. Aunque realmente las predicciones se podrían hacer usando todos los datos, y no solo el train.

Los parámetros principales, los cuales son: número de vecinos, proporción de entrenamiento y número de recomendaciones, están definidos como constantes al inicio del fichero, lo que permite ajustarlos fácilmente sin modificar la lógica del programa.

Se ejecutó la evaluación del modelo dividiendo las valoraciones de cada usuario en un 80% para entrenamiento y un 20% para test, utilizando $k = 20$ vecinos. El resultado obtenido fue un MAE (error absoluto medio) de 0.8388, lo que significa que, de media, la predicción del modelo se desvía en aproximadamente 0.84 puntos respecto a la puntuación real, en una escala de 1 a 5.

```
=====
RESULTADOS DE LA EVALUACION
=====
Numero de vecinos (k): 20
Predicciones realizadas: 27489
Predicciones no posibles (sin vecinos): 84541
MAE (error absoluto medio): 0.8388

Interpretacion: de media, la prediccion se desvia 0.84 puntos
respecto a la puntuacion real (en una escala de 1 a 5).
=====

=====
MODELO DE MACHINE LEARNING - FILTRADO COLABORATIVO
=====
```

Este resultado se considera razonable para un modelo de filtrado colaborativo básico. Hay que tener en cuenta que un número elevado de predicciones (84 541) no pudieron realizarse porque el usuario de test no tenía vecinos suficientes que hubieran puntuado ese artículo concreto. Esto es esperable en un sistema con una matriz usuario-artículo muy dispersa, donde la mayoría de usuarios solo ha valorado una pequeña fracción de los artículos disponibles. A pesar de esta limitación, las 27 489 predicciones que sí se completaron muestran que el modelo es capaz de aproximar las preferencias de los usuarios con un error inferior a un punto.

6.2. Segundo Opcional: mejora en visualización

Se ha desarrollado un fichero independiente llamado `dashboard.py` que ofrece una interfaz gráfica interactiva como alternativa al menú por terminal de `menu_visualizacion.py`. El objetivo es permitir al usuario explorar las mismas visualizaciones de la segunda parte del proyecto de forma más cómoda e intuitiva, sin necesidad de escribir opciones por consola.

La interfaz se ha construido con Tkinter, que forma parte de la biblioteca estándar de Python y no requiere instalación adicional. La ventana se divide en dos zonas: un panel izquierdo con los controles de selección y un panel derecho donde se muestra la gráfica generada. Los controles incluyen un desplegable para elegir la visualización, otro para seleccionar la categoría, un campo de texto para introducir el ASIN de un artículo concreto (en el caso del histograma por nota) y un selector de modo para la evolución temporal. Un botón central ejecuta la consulta y dibuja el resultado directamente dentro de la ventana, sin abrir ventanas externas de `matplotlib`.

Las siete visualizaciones disponibles son exactamente las mismas que ofrece el menú por terminal: reviews por año, popularidad de artículos, histograma por nota, evolución temporal

acumulada, histograma de reviews por usuario, nube de palabras por categoría y media de nota por categoría. Las consultas SQL y las consultas a MongoDB no se han duplicado: `dashboard.py` importa directamente las funciones de consulta definidas en `menu_visualizacion.py`, de modo que ambos ficheros comparten la misma lógica de acceso a datos y cualquier corrección futura en las consultas se refleja automáticamente en los dos.

Esta solución permite mantener el menú original por terminal completamente funcional, ya que no se ha modificado ninguna línea de `menu_visualizacion.py` ni de ningún otro fichero del proyecto. El usuario puede elegir entre ejecutar `python menu_visualizacion.py` para la versión por terminal o `python dashboard.py` para la versión gráfica interactiva.

7. División del trabajo

El proyecto se ha realizado en pareja. A continuación se detalla el reparto de tareas entre los dos estudiantes:

Rodrigo Alejandro Sicilia Maroto se encargó principalmente del diseño y la implementación del backend de datos. En la primera parte, diseñó el modelo relacional de MySQL, realizó el análisis exploratorio de los ficheros JSON mediante scripts auxiliares en Python para validar las hipótesis del diseño (detección de artículos multicategoría, revisión de duplicados, análisis de `reviewerName`), e implementó los ficheros `load_data.py` y `configuracion.py`. En la tercera parte, desarrolló el fichero `neo4JProyecto.py` completo, implementando los cuatro apartados del enunciado (similitudes entre usuarios mediante correlación de Pearson, enlaces entre usuarios y artículos, usuarios multicategoría, y artículos populares con artículos en común). En la cuarta parte, implementó `inserta_dataset.py` para la inserción del nuevo dataset. En la quinta parte, implementó el mecanismo de recomendación en Python que, dado un usuario y un tipo de artículo, devuelve los 10 artículos más populares que ese usuario no ha consumido.

Claudia Moya Rodríguez se encargó principalmente del desarrollo de las visualizaciones y de la redacción del informe. En la primera parte, participó en la validación del diseño y redactó la documentación del modelo. En la segunda parte, desarrolló el fichero `menu_visualizacion.py` completo, implementando las siete opciones de visualización del menú (evolución por años, popularidad de artículos, histograma por nota, evolución temporal, histograma por usuario, nube de palabras y visualización extra), así como toda la lógica del menú por terminal. En la quinta parte, elaboró las tres visualizaciones en PowerBI a partir de los datos exportados desde MySQL, recreando los gráficos de evolución por años, popularidad de artículos y media de nota por categoría. Además, redactó la mayor parte del informe del proyecto.

Ambos estudiantes participaron conjuntamente en las decisiones de diseño más relevantes, especialmente en la elección de `REVIEW_CATEGORIA` como tabla de procedencia categórica, en la revisión y corrección del código, y en la preparación de la entrega final.

Por otro lado, las partes opcionales y las conclusiones se hicieron de manera conjunta.

8. Conclusión

Este proyecto ha permitido trabajar de forma práctica con tres tecnologías de bases de datos distintas, MySQL, MongoDB y Neo4J, aplicadas sobre un mismo conjunto de datos reales. A lo largo del desarrollo se han afrontado problemas concretos de diseño, como la multicategoría de los artículos o la deduplicación de reviews entre ficheros, que han

obligado a tomar decisiones justificadas y a validarlas empíricamente sobre los datos originales.

Además del diseño y la carga, se ha experimentado con la visualización de datos tanto desde Python como desde Power BI, con la representación en grafos mediante Neo4J y con la propuesta de un modelo de recomendación basado en filtrado colaborativo. Todo ello ha servido para entender que cada tecnología tiene sus fortalezas: MySQL para la estructura y las consultas relacionales, MongoDB para la información documental y flexible, y Neo4J para representar visualmente las relaciones entre entidades.

En conjunto, el proyecto refleja un flujo completo de trabajo con datos: desde su almacenamiento y organización hasta su análisis, visualización y uso para recomendaciones.